

DEVELOPER HANDBOOK

BusinessHub

Software Requirements Specification

Everything a development team needs to build release 1 without further arbitration: a frozen stack, a complete data model, an endpoint-by-endpoint API catalogue, a permission matrix, state machines, and user stories with testable acceptance criteria.

Attribute	Value
Document type	Software requirements specification (technical scope of work)
Companion document	BusinessHub — Platform Design Dossier
Scope	Release 1 — web application, mobile application, REST API
Audience	Development team, QA, DevOps, technical lead, product owner
Stack	Next.js · Flutter · NestJS · PostgreSQL 16 · Redis 7 · Docker
Status	Build contract — version 1.0
Language	English

Table of Contents

1. Purpose, Scope and How to Read This Document	6
1.1 In Scope for Release 1	6
1.2 Out of Scope for Release 1	6
1.3 Definition of Terms	6
1.4 Reading Order for a New Developer	7
2. Frozen Technology Stack	8
2.1 Repository Layout	9
2.2 Coding Standards	9
3. System Architecture	10
3.1 Runtime Components	10
3.2 Module Boundaries Inside the API	11
3.3 Cross-Cutting Request Pipeline	11
4. Environments, Configuration and Local Setup	13
4.1 Environments	13
4.2 Configuration Contract	13
4.3 Getting the Stack Running Locally	14
4.4 Definition of Ready for a Ticket	14
5. Identity, Tenancy and Access Control	15
5.1 Authentication	15
5.2 Tenancy Model	15
5.3 Permission Catalogue	15
5.4 Role / Permission Matrix	16
5.5 Record-Level Rules	17
6. Data Model	18
6.1 Conventions	18
6.2 Enumerations	18
6.3 Identity and Tenancy Tables	19
6.4 CRM and Case Tables	20
6.5 Workflow Tables	21
6.6 Document, Finance, Messaging and Audit Tables	22
6.7 Retention and Deletion	24
7. API Design Conventions	25
7.1 Base Rules	25
7.2 Response Envelope	25
7.3 Status Codes and Error Codes	26
7.4 Filtering, Sorting, Pagination and Search	26

7.5 Concurrency, Webhooks and Rate Limits	27
8. Endpoint Catalogue.....	28
8.1 Authentication and Session	28
8.2 Organisations, Users and Roles.....	28
8.3 CRM.....	29
8.4 Cases and Requests.....	29
8.5 Workflow — Tasks and Approvals	30
8.6 Documents.....	31
8.7 Finance.....	32
8.8 Communication, Reporting and Administration	32
9. Functional Specification by Module.....	34
M1 — Accounts, Organisations and Access	34
Functional Requirements.....	34
User Stories.....	34
M2 — CRM: Clients, Partners and Contacts.....	35
Functional Requirements.....	35
User Stories.....	36
M3 — Requests and Cases.....	37
Functional Requirements.....	37
User Stories.....	37
M4 — Workflow: Tasks, Approvals and Escalations	38
Functional Requirements.....	38
User Stories.....	39
M5 — Documents	39
Functional Requirements.....	39
User Stories.....	40
M6 — Finance	40
Functional Requirements.....	40
User Stories.....	41
M7 — Communication and Notifications	42
Functional Requirements.....	42
User Stories.....	42
M8 — Reporting and Dashboards.....	43
Functional Requirements.....	43
User Stories.....	43
M9 — Administration, Audit and Integrations	44
Functional Requirements.....	44
User Stories.....	44

10. State Machines and Process Rules	46
10.1 Case Transition Table	46
10.2 Invoice and Payment State Rules	47
10.3 SLA Computation	47
11. Cross-Cutting Business Rules	49
12. Web Application Specification	50
12.1 Route Map	50
12.2 Screen Composition Rules	50
12.3 Accessibility and Internationalisation	51
12.4 Web Performance Budget.....	51
13. Mobile Application Specification	52
13.1 Scope.....	52
13.2 Offline Behaviour	52
13.3 Mobile Platform Requirements	52
14. External Integrations.....	54
14.1 Integration Contracts	54
14.2 Webhook Handling Rules	54
15. Non-Functional Requirements	55
15.1 Performance and Capacity	55
15.2 Availability and Resilience.....	55
15.3 Security Requirements	55
15.4 Privacy and Compliance	56
16. Observability	57
17. Test Strategy	58
17.1 Test Pyramid and Coverage Gates	58
17.2 Mandatory End-to-End Journeys	58
17.3 Test Data and Environments.....	58
18. Delivery, CI/CD and Operations	60
18.1 Pipeline	60
18.2 Migration Discipline	60
18.3 Release Roadmap.....	60
18.4 Definition of Done.....	61
19. Risks and Mitigations	62
20. Appendices.....	63
A. Notification Event Catalogue	63
B. Standard Audit Actions	63
C. Seed Data Loaded in Local and CI Environments	64
D. Open Decisions Required Before PO Ends	64

E. Glossary of Technical Terms.....	64
-------------------------------------	----

1. Purpose, Scope and How to Read This Document

This document is the build contract for BusinessHub. It translates the design dossier into decisions a development team can implement without further arbitration: a frozen technology stack, a complete relational schema, an endpoint-by-endpoint API catalogue, a permission matrix, state machines, and user stories with testable acceptance criteria.

Anything stated here as **MUST** is a hard requirement and is covered by an acceptance test. **SHOULD** marks a strong recommendation that may be traded away with the product owner's written agreement. **MAY** marks an option left to the implementing team.

1.1 In Scope for Release 1

- Nine functional modules: accounts and organisations, CRM, requests and cases, workflow, documents, finance, communication, reporting, administration.
- Three delivery surfaces: a Next.js web application (internal consoles plus client/partner portal), a Flutter mobile application, and a documented public REST API.
- Multi-organisation tenancy with role-based access control, full audit trail and data isolation.
- Four outbound integration families: payment, messaging (email/SMS/push), e-signature and object storage.

1.2 Out of Scope for Release 1

- Machine-learning recommendations, predictive scoring and anomaly detection.
- Public marketplace and product catalogue.
- Sector-specific connectors beyond the four integration families above.
- Native desktop clients and offline-first web usage (the mobile app has a bounded offline queue only).

1.3 Definition of Terms

Term	Meaning in this document
Organisation	Any tenant entity: the operating company itself, a client company, or a partner company. The unit of data isolation.
Membership	The link between a user and an organisation, carrying exactly one role. A user may hold several memberships.
Case	The central business object, also called a request or file. Everything (tasks, documents, invoices, messages) hangs off a case.
Task	A unit of work inside a case, assigned to one user, with a type, a due date and a status.
Approval	A decision attached to a task, recorded with the deciding user, the level, the outcome and a timestamp.
SLA	The contractual deadline attached to a case or task, computed from business calendars, and the source of reminders and escalations.
Actor	The authenticated principal performing an action; recorded on every audit event. May be a user or a service account.

1.4 Reading Order for a New Developer

1. Sections 2 and 3 — stack, repository layout and how to run the system locally.
2. Section 5 — identity, tenancy and the permission model, because every endpoint depends on it.
3. Section 6 — the relational schema; the DDL is authoritative.
4. Section 7 and 8 — API conventions, then the endpoint catalogue for the module you are assigned.
5. Section 9 — the user stories and acceptance criteria for that module; these become your tests.

2. Frozen Technology Stack

The design dossier left several technology choices open. They are now closed. Substituting a component listed below requires a written architecture decision record and product owner approval; it is not a developer-level choice.

Layer	Decision	Version floor	Rationale
Web front end	Next.js (App Router) + TypeScript	14.2	Server components for portal SEO and first paint; a single React codebase for portal and consoles.
Web UI kit	Tailwind CSS + shadcn/ui + TanStack Query	3.4 / current	Accessible primitives, no runtime CSS cost, predictable server-state caching.
Mobile	Flutter + Dart	3.22	One codebase for Android and iOS; strong offline storage and background-task support.
Mobile state / storage	Riverpod + Drift (SQLite) + flutter_secure_storage	current	Testable state layer, relational offline queue, OS-keychain token storage.
Backend	NestJS + TypeScript on Node.js	10 / Node 20 LTS	Modular by construction, first-class DI, mature OpenAPI and validation tooling.
ORM / migrations	Prisma	5.x	Type-safe access, deterministic migration history, generated client shared with tests.
Database	PostgreSQL	16	Transactional integrity, JSONB for flexible payloads, row-level security for tenancy.
Cache / queue	Redis 7 + BullMQ	7 / current	Sessions, rate limits, idempotency keys, and durable background jobs.
Object storage	S3-compatible (AWS S3 or MinIO)	API 2006-03-01	Documents never transit the API server; pre-signed URLs both ways.
Authentication	OAuth 2.0 / OIDC, JWT access + rotating refresh, TOTP MFA	RFC 6749 / 6238	Standards-based, works identically for web, mobile and machine clients.
Search	PostgreSQL full-text (tsvector) in R1	16	Avoids a second datastore until volume justifies OpenSearch.
Containers / CI	Docker + Docker Compose, GitHub Actions	current	Identical images from laptop to production; pipeline as code.
Observability	OpenTelemetry → Grafana stack (Loki, Tempo, Prometheus)	current	One instrumentation standard for logs, traces and metrics.
Error tracking	Sentry (web, mobile and backend SDKs)	current	Release-tagged stack traces across all three surfaces.

VERSIONING Every version floor above is a minimum, not a pin. Dependency upgrades within a major version are routine; a major-version jump is an architecture decision record.

2.1 Repository Layout

A single monorepo managed with pnpm workspaces and Turborepo. Flutter lives inside the same repository but outside the pnpm workspace graph.

businesshub/	
├── apps/	
│ ├── api/	NestJS backend (the only writer to PostgreSQL)
│ ├── web/	Next.js – consoles + client/partner portal
│ ├── worker/	BullMQ consumers: notifications, exports, SLA sweeps
│ └── mobile/	Flutter application (Android + iOS)
├── packages/	
│ ├── contracts/	OpenAPI source of truth + generated TS types
│ ├── permissions/	Permission codes + RBAC matrix, imported by api and web
│ ├── ui/	Shared React components and design tokens
│ └── config/	ESLint, TypeScript, Prettier, Tailwind presets
├── infra/	
│ ├── docker/	Dockerfiles + docker-compose.yml (dev + CI)
│ ├── migrations/	Prisma migration history (never edited after merge)
│ └── terraform/	Cloud infrastructure as code
├── docs/	ADRs, runbooks, this specification
└── .github/workflows/	CI pipelines

2.2 Coding Standards

- TypeScript `strict` mode is on everywhere; `any` requires an inline justification comment.
- ESLint + Prettier run in a pre-commit hook and again in CI; CI fails on any lint error.
- Commits follow Conventional Commits; the changelog and semantic version are generated from them.
- Branch naming: `feat/<ticket>-<slug>`, `fix/<ticket>-<slug>`. Direct pushes to `main` are blocked.
- Every pull request requires one approving review, green CI, and no drop in coverage below the thresholds in section 17.
- Dart code follows `flutter_lints` with `prefer_const` and `always_declare_return_types` enabled.

3. System Architecture

BUSINESSHUB — LOGICAL ARCHITECTURE

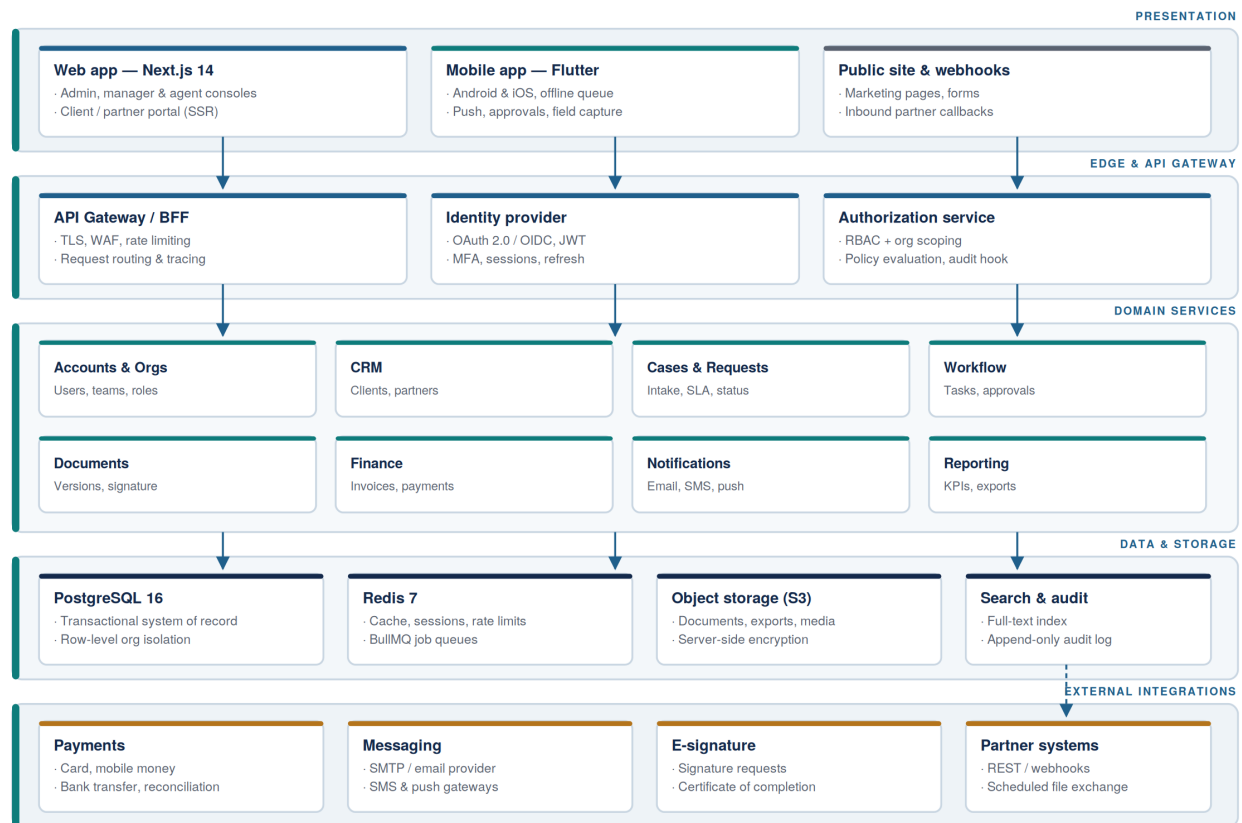


Figure 1 — Logical architecture, with the frozen stack applied

3.1 Runtime Components

Component	Responsibility	Scaling model
api	All business logic, authorisation, validation and database writes. Stateless.	Horizontal; 2 replicas minimum in production.
web	Server-rendered pages and the browser bundle. Calls api only over HTTPS.	Horizontal behind a CDN.
worker	Queue consumers: notification delivery, report generation, SLA sweeps, webhook dispatch, retention jobs.	Horizontal by queue; concurrency configured per queue.
postgres	System of record. One primary, one hot standby.	Vertical + read replicas for reporting in R2.
redis	Sessions, cache, rate-limit counters, idempotency keys, BullMQ queues.	Single primary with replica; separate logical databases per concern.
storage	Object storage for documents, exports and generated PDFs.	Managed service; no application-side sharding.

3.2 Module Boundaries Inside the API

The backend is a modular monolith, not a set of microservices. Each module owns its tables and exposes a service interface; cross-module access goes through that interface, never through another module's repository. This keeps a future extraction into separate services cheap without paying distributed-system costs in release 1.

```
apps/api/src/
├── modules/
│   ├── identity/      users, sessions, MFA, password reset, service accounts
│   ├── tenancy/       organisations, memberships, roles, invitations
│   ├── crm/           client & partner records, contacts, segments, consents
│   ├── cases/         requests, case types, SLA computation, comments
│   ├── workflow/      tasks, approvals, escalations, process templates
│   ├── documents/     upload sessions, versions, classification, e-signature
│   ├── finance/       quotations, invoices, payments, reconciliation
│   ├── messaging/     notification templates, channels, preferences, delivery
│   ├── reporting/     KPI queries, saved views, exports
│   └── admin/         reference data, settings, integrations, audit read API
├── common/            guards, interceptors, filters, pipes, decorators
├── infrastructure/    prisma client, redis, s3, mail, sms, payment adapters
└── main.ts
```

3.3 Cross-Cutting Request Pipeline

Every inbound request passes through the same ordered chain. Implement it once, in `common/`, and never bypass it in a controller.

#	Stage	Behaviour	Failure response
1	Correlation	Assign or propagate <code>X-Request-Id</code> ; open an OpenTelemetry span.	—
2	Rate limiting	Redis sliding window, keyed by IP for anonymous and by user for authenticated calls.	429 with <code>Retry-After</code>
3	Authentication	Verify the JWT signature, expiry, issuer and audience; load the session.	401 <code>AUTH_REQUIRED</code>
4	Tenancy resolution	Resolve the active organisation from <code>X-Organization-Id</code> and confirm the caller holds a membership in it.	403 <code>ORG_FORBIDDEN</code>
5	Authorisation	Evaluate the required permission code against the membership role and any record-level rule.	403 <code>PERMISSION_DENIED</code>
6	Validation	Validate and coerce the payload with class-validator DTOs; reject unknown properties.	422 <code>VALIDATION_FAILED</code>
7	Idempotency	For POST/PATCH/DELETE with an <code>Idempotency-Key</code> , replay the stored response if the key is known.	200 replay, or 409 on key reuse with a different body
8	Handler	Execute business logic inside a database transaction where more than one table is written.	Mapped error codes
9	Audit	Emit an audit event for every mutating action, with before and after snapshots.	Never blocks the response; failures alert
10	Serialisation	Strip fields the caller is not allowed to see, then shape the envelope.	—

SECURITY DEFAULT Stages 3 to 5 are enforced by a global guard. A controller that needs to be public **MUST** opt out explicitly with `@Public()` — there is no implicit public route.

4. Environments, Configuration and Local Setup

4.1 Environments

Environment	Purpose	Data	Deployment trigger
local	Developer laptop, full stack in Docker Compose.	Seeded fixtures only.	Manual.
ci	Ephemeral, created per pipeline run and destroyed after.	Fresh migrations + fixtures.	Every push.
staging	Pre-production rehearsal, identical topology to production.	Anonymised copy of production, refreshed weekly.	Automatic on merge to main .
production	Live service.	Real data.	Manual promotion of a staging build, with approval.

4.2 Configuration Contract

Configuration is read from environment variables only — never from files committed to the repository. The API validates the whole environment at boot with a Zod schema and refuses to start if a required variable is missing or malformed. Secrets come from the cloud secret manager and are injected at deploy time.

```
# --- core -----
NODE_ENV=production                # local | ci | staging | production
PORT=3000
PUBLIC_API_URL=https://api.businesshub.app
PUBLIC_WEB_URL=https://businesshub.app

# --- data -----
DATABASE_URL=postgresql://user:pass@host:5432/businesshub?schema=public
DATABASE_POOL_MAX=20
REDIS_URL=redis://host:6379

# --- identity -----
JWT_PRIVATE_KEY=<PEM, RS256>      # access token signing, rotated quarterly
JWT_PUBLIC_KEY=<PEM>
JWT_ACCESS_TTL=900                # seconds (15 min)
JWT_REFRESH_TTL=2592000           # seconds (30 days)
MFA_ISSUER=BusinessHub
PASSWORD_PEPPER=<32-byte secret>

# --- storage -----
S3_ENDPOINT=https://s3.eu-west-1.amazonaws.com
S3_BUCKET=businesshub-documents
S3_REGION=eu-west-1
S3_ACCESS_KEY_ID=<secret>
S3_SECRET_ACCESS_KEY=<secret>
S3_PRESIGN_TTL=600               # seconds

# --- integrations -----
PAYMENT_PROVIDER=stripe           # stripe | flutterwave | manual
PAYMENT_API_KEY=<secret>
PAYMENT_WEBHOOK_SECRET=<secret>
MAIL_PROVIDER=ses                 # ses | sendgrid | smtp
MAIL_FROM="BusinessHub <no-reply@businesshub.app>"
SMS_PROVIDER=twilio
SMS_API_KEY=<secret>
PUSH_FCM_CREDENTIALS=<json, base64>
ESIGN_PROVIDER=docusign           # docusign | none
```

```
# --- observability -----
OTEL_EXPORTER_OTLP_ENDPOINT=https://otel.internal:4318
SENTRY_DSN=<dsn>
LOG_LEVEL=info                                # debug | info | warn | error
```

4.3 Getting the Stack Running Locally

```
git clone git@github.com:<org>/businesshub.git && cd businesshub
cp .env.example .env                # dev defaults; no real secrets required
pnpm install
docker compose up -d postgres redis minio
pnpm --filter api prisma migrate deploy
pnpm --filter api prisma db seed      # roles, case types, demo tenants, users
pnpm dev                             # api :3000, web :3001, worker attached

# mobile, in a second shell
cd apps/mobile && flutter pub get && flutter run --dart-define=API_URL=http://10.0.2.2:3000
```

The seed creates one operating company, two client organisations, one partner organisation, and one user per role with the password **Password123!**. Seed data **MUST** never be loaded in staging or production; the seed script aborts unless `NODE_ENV` is `local` or `ci`.

4.4 Definition of Ready for a Ticket

- The user story references a section of this document and names the permission code it exercises.
- Acceptance criteria are written as Given/When/Then and are automatable.
- API changes are reflected in `packages/contracts` before implementation starts.
- Any new table or column has a migration reviewed alongside the code.
- The ticket states whether an audit event, a notification, or both, are expected.

5. Identity, Tenancy and Access Control

5.1 Authentication

Concern	Requirement
Password storage	Argon2id, memory 64 MiB, iterations 3, parallelism 4, plus a server-side pepper held in the secret manager.
Password policy	Minimum 12 characters; rejected if present in the breached-password list (k-anonymity check against an offline dataset). No forced periodic rotation.
Access token	JWT, RS256, 15-minute lifetime. Claims: <code>sub</code> , <code>sid</code> , <code>orgs[]</code> , <code>iat</code> , <code>exp</code> , <code>iss</code> , <code>aud</code> . It carries no permission list — permissions are resolved server-side on every call.
Refresh token	Opaque 256-bit value, hashed in the database, 30-day lifetime, single use with rotation. Reuse of a consumed token revokes the entire token family and raises a security alert.
MFA	TOTP (RFC 6238), 30-second step, ± 1 window tolerance. Mandatory for Administrator, Super-administrator and Manager. Ten single-use recovery codes are issued at enrolment and shown exactly once.
Brute-force defence	Five failed attempts within 15 minutes locks the account for 15 minutes and emails the owner. Counters are per account and per source IP.
Sessions	Listed per user with device, IP, location and last-seen. A user may revoke any of their sessions; an administrator may revoke any session in their organisation.
Service accounts	Machine clients use the OAuth 2.0 client-credentials grant. Secrets are shown once, are rotatable, and are scoped to a single organisation and a fixed permission set.

5.2 Tenancy Model

Every business row carries an `organization_id`. Isolation is enforced at two levels, deliberately redundant: the application always filters by the active organisation, and PostgreSQL row-level security refuses cross-tenant reads even if application code is wrong.

```
-- Applied to every tenant-scoped table
ALTER TABLE "case" ENABLE ROW LEVEL SECURITY;

CREATE POLICY case_tenant_isolation ON "case"
  USING (organization_id = current_setting('app.current_org')::uuid);

-- The API sets the variable at the start of every transaction:
--   SET LOCAL app.current_org = '<uuid>';
-- Requests that legitimately span organisations (super-admin reporting)
-- run as a role that bypasses RLS and are always audited.
```

5.3 Permission Catalogue

Permissions are string codes of the form `<resource>.<action>`. They live in `packages/permissions` and are imported by both the API guard and the web UI, so a hidden button and a rejected request always agree.

Resource	Permission codes
User	user.read, user.invite, user.update, user.deactivate, user.impersonate
Organisation	org.read, org.create, org.update, org.suspend, org.settings.manage
Role	role.read, role.assign, role.manage
Client / partner	crm.read, crm.create, crm.update, crm.delete, crm.export
Case	case.read.own, case.read.org, case.read.all, case.create, case.update, case.assign, case.transition, case.close, case.reopen, case.delete
Task	task.read, task.create, task.assign, task.complete, task.reassign
Approval	approval.read, approval.decide, approval.override
Document	document.read, document.upload, document.download, document.classify, document.delete, document.sign.request
Finance	finance.read, invoice.create, invoice.issue, invoice.void, payment.record, payment.refund, finance.reconcile
Notification	notification.read, notification.template.manage, notification.send.bulk
Report	report.read, report.export, report.manage
Admin	admin.reference.manage, admin.integration.manage, admin.audit.read, admin.impersonate

5.4 Role / Permission Matrix

Legend: ● full · ◐ limited to records the user or their organisation owns · ○ none. Roles are seeded as system roles and cannot be deleted; an administrator MAY create additional custom roles from the same permission codes.

Permission group	Visitor	Client	Partner	Agent	Manager	Admin	Super
user.read / invite / update	○○○	◐○○	◐●◐	◐○○	●●◐	●●●	●●●
user.deactivate / impersonate	○○	○○	◐○	○○	◐○	●○	●●
org.read / update / settings	○○○	◐◐○	◐●◐	◐○○	●◐○	●●●	●●●
role.read / assign / manage	○○○	○○○	◐◐○	○○○	●◐○	●●●	●●●
crm.read / create / update	○○○	◐○○	◐○○	●●●	●●●	●●●	●●●
crm.delete / export	○○	○○	○○	○◐	◐●	●●	●●
case.create / read	◐○	●◐	●◐	●●	●●	●●	●●
case.assign / transition	○○	○○	○○	◐●	●●	●●	●●
case.close / reopen / delete	○○○	○○○	○○○	●○○	●○○	●●●	●●●
task.read / complete / reassign	○○○	○○○	◐○○	◐○○	●●●	●●●	●●●
approval.decide / override	○○	○○	○○	○○	●○	●●	●●

Permission group	Visitor	Client	Partner	Agent	Manager	Admin	Super
document.upload / download	oo	●●	●●	●●	●●	●●	●●
document.classify / delete	oo	oo	oo	●o	●●	●●	●●
finance.read / invoice.issue	oo	●o	●o	●o	●●	●●	●●
payment.record / refund	oo	oo	oo	oo	●o	●●	●●
report.read / export / manage	ooo	●●o	●●o	●●o	●●●	●●●	●●●
admin.* (reference, integration, audit)	o	o	o	o	●	●	●

5.5 Record-Level Rules

Role permissions are necessary but not sufficient. After the permission check passes, the following record-level predicates MUST also hold. They are implemented as a reusable `CaseAbility` / `DocumentAbility` service, not copied into controllers.

Rule	Statement
R-01 Tenant	The record's <code>organization_id</code> equals the active organisation, unless the caller holds <code>case.read.all</code> (Super-administrator only).
R-02 Ownership	A holder of <code>case.read.own</code> sees only cases where they are the owner, the creator, or an assignee on a task.
R-03 Client visibility	A Client sees a case only if their organisation is the case's client organisation, and never sees internal comments or internal-only tasks.
R-04 Confidentiality	A document classified <code>confidential</code> is readable only by users holding <code>document.classify</code> , plus the case owner.
R-05 Segregation of duties	The user who created an invoice MUST NOT be the user who approves it, and MUST NOT be the user who records its payment, unless the organisation has fewer than three active finance users, in which case the override is recorded as an audit event.
R-06 Approval level	A user may only decide approvals at or below their approval level; an approval above the level requires escalation, never a silent pass.
R-07 Closed records	A case in <code>CLOSED</code> or <code>ARCHIVED</code> rejects every mutation except <code>case.reopen</code> and audit reads.
R-08 Impersonation	While impersonating, all write permissions are denied; the session is read-only and every read is audited with both the real and the effective actor.

6. Data Model

CORE DATA MODEL — ENTITY RELATIONSHIPS

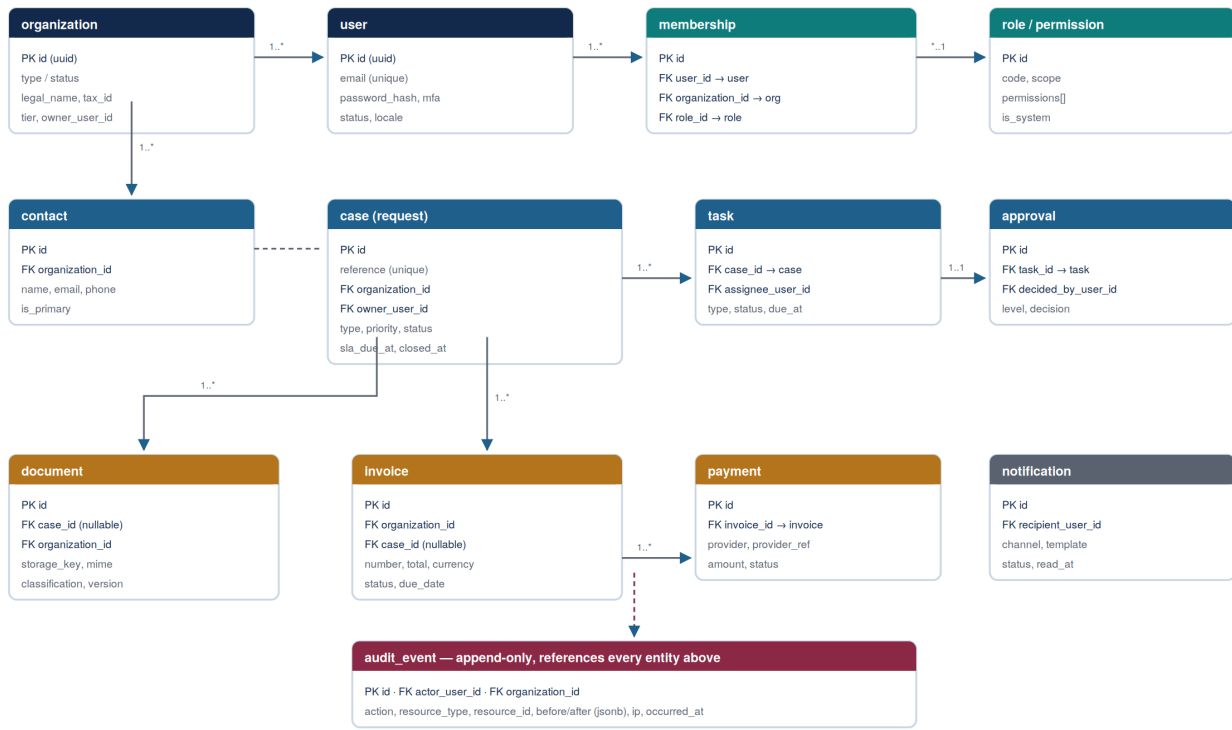


Figure 2 — Core entities and their relationships

6.1 Conventions

- Primary keys are UUID v7 (time-ordered), generated by the application, never by a sequence.
- Every table carries `created_at`, `updated_at` (both `timestampz`), `created_by` and `updated_by`.
- Deletion is logical: `deleted_at timestampz NULL`. Physical deletion happens only through the retention job.
- Money is stored as `numeric(14,2)` plus an ISO-4217 `currency char(3)`. Never a float.
- Enumerations are PostgreSQL enum types so that an invalid value cannot be written.
- Every foreign key is indexed. Every tenant-scoped table has a composite index leading with `organization_id`.
- Free-form business payloads use `jsonb` with a validated shape at the application layer, never `text` holding JSON.

6.2 Enumerations

```

CREATE TYPE org_type AS ENUM ('INTERNAL', 'CLIENT', 'PARTNER');
CREATE TYPE org_status AS ENUM ('PENDING', 'ACTIVE', 'SUSPENDED', 'CLOSED');
CREATE TYPE user_status AS ENUM ('INVITED', 'ACTIVE', 'LOCKED', 'DISABLED');
CREATE TYPE case_status AS ENUM ('DRAFT', 'SUBMITTED', 'QUALIFIED', 'ASSIGNED', 'IN_PROGRESS',
    'AWAITING_INFO', 'ON_HOLD', 'IN_REVIEW', 'ESCALATED',
    'APPROVED', 'REJECTED', 'RESOLVED', 'CLOSED', 'ARCHIVED');
CREATE TYPE case_priority AS ENUM ('LOW', 'NORMAL', 'HIGH', 'CRITICAL');
CREATE TYPE task_status AS ENUM ('OPEN', 'IN_PROGRESS', 'BLOCKED', 'DONE', 'CANCELLED');
CREATE TYPE approval_state AS ENUM ('PENDING', 'APPROVED', 'REJECTED', 'DELEGATED', 'EXPIRED');
CREATE TYPE doc_class AS ENUM ('PUBLIC', 'INTERNAL', 'RESTRICTED', 'CONFIDENTIAL');
CREATE TYPE invoice_status AS ENUM ('DRAFT', 'ISSUED', 'PARTIALLY_PAID', 'PAID', 'OVERDUE',
    'DISPUTED', 'CREDITED', 'VOID');
CREATE TYPE payment_status AS ENUM ('INITIATED', 'PENDING', 'SUCCEEDED', 'FAILED', 'REFUNDED');
  
```

```
CREATE TYPE notif_channel AS ENUM ('IN_APP','EMAIL','SMS','PUSH');
CREATE TYPE notif_status AS ENUM ('QUEUED','SENT','DELIVERED','FAILED','READ');
```

6.3 Identity and Tenancy Tables

```
CREATE TABLE organization (
  id                uuid PRIMARY KEY,
  type              org_type NOT NULL,
  status            org_status NOT NULL DEFAULT 'PENDING',
  legal_name        text NOT NULL,
  trading_name      text,
  registration_no    text,
  tax_id            text,
  tier              text, -- commercial segmentation
  country           char(2) NOT NULL,
  address           jsonb,
  owner_user_id     uuid REFERENCES "user"(id), -- account manager
  parent_id         uuid REFERENCES organization(id),
  settings          jsonb NOT NULL DEFAULT '{}',
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now(),
  deleted_at        timestampz,
  CONSTRAINT org_tax_unique UNIQUE NULLS NOT DISTINCT (country, tax_id)
);
CREATE INDEX ix_org_status ON organization(status) WHERE deleted_at IS NULL;
CREATE INDEX ix_org_name_fts ON organization USING gin (to_tsvector('simple', legal_name));

CREATE TABLE "user" (
  id                uuid PRIMARY KEY,
  email             citext NOT NULL UNIQUE,
  email_verified_at timestampz,
  password_hash     text, -- null for SSO-only accounts
  first_name        text NOT NULL,
  last_name         text NOT NULL,
  phone            text,
  locale           text NOT NULL DEFAULT 'en',
  timezone          text NOT NULL DEFAULT 'UTC',
  status            user_status NOT NULL DEFAULT 'INVITED',
  mfa_secret        text, -- encrypted at rest
  mfa_enabled_at    timestampz,
  last_login_at     timestampz,
  failed_attempts   int NOT NULL DEFAULT 0,
  locked_until      timestampz,
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now(),
  deleted_at        timestampz
);

CREATE TABLE role (
  id                uuid PRIMARY KEY,
  organization_id    uuid REFERENCES organization(id), -- NULL = system role
  code              text NOT NULL,
  name              text NOT NULL,
  permissions        text[] NOT NULL DEFAULT '{}',
  approval_level    int NOT NULL DEFAULT 0,
  is_system         boolean NOT NULL DEFAULT false,
  created_at        timestampz NOT NULL DEFAULT now(),
  UNIQUE NULLS NOT DISTINCT (organization_id, code)
);

CREATE TABLE membership (
  id                uuid PRIMARY KEY,
  user_id           uuid NOT NULL REFERENCES "user"(id) ON DELETE CASCADE,
  organization_id    uuid NOT NULL REFERENCES organization(id) ON DELETE CASCADE,
  role_id           uuid NOT NULL REFERENCES role(id),
  is_default        boolean NOT NULL DEFAULT false,
  invited_by        uuid REFERENCES "user"(id),
  accepted_at       timestampz,
  created_at        timestampz NOT NULL DEFAULT now(),
  deleted_at        timestampz,
  UNIQUE (user_id, organization_id)
);
```

```

CREATE INDEX ix_membership_org ON membership(organization_id) WHERE deleted_at IS NULL;

CREATE TABLE session (
  id                uuid PRIMARY KEY,
  user_id           uuid NOT NULL REFERENCES "user"(id) ON DELETE CASCADE,
  refresh_token_hash text NOT NULL,
  family_id         uuid NOT NULL,          -- rotation family
  user_agent        text,
  ip               inet,
  expires_at        timestampz NOT NULL,
  revoked_at        timestampz,
  created_at        timestampz NOT NULL DEFAULT now()
);
CREATE INDEX ix_session_user ON session(user_id) WHERE revoked_at IS NULL;

```

6.4 CRM and Case Tables

```

CREATE TABLE contact (
  id                uuid PRIMARY KEY,
  organization_id   uuid NOT NULL REFERENCES organization(id) ON DELETE CASCADE,
  user_id           uuid REFERENCES "user"(id),  -- set when the contact has portal access
  first_name        text NOT NULL,
  last_name         text NOT NULL,
  email            citext,
  phone            text,
  job_title         text,
  is_primary        boolean NOT NULL DEFAULT false,
  consents          jsonb NOT NULL DEFAULT '{}', -- {marketing:bool, sms:bool, at:iso}
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now(),
  deleted_at        timestampz
);
CREATE UNIQUE INDEX ux_contact_primary ON contact(organization_id)
  WHERE is_primary AND deleted_at IS NULL;

CREATE TABLE case_type (
  id                uuid PRIMARY KEY,
  code              text NOT NULL UNIQUE,
  name              text NOT NULL,
  description        text,
  form_schema        jsonb NOT NULL,          -- JSON Schema for the intake form
  required_docs      text[] NOT NULL DEFAULT '{}',
  sla_hours          int NOT NULL DEFAULT 72,
  approval_levels    int NOT NULL DEFAULT 1,
  is_client_visible boolean NOT NULL DEFAULT true,
  is_active          boolean NOT NULL DEFAULT true
);

CREATE TABLE "case" (
  id                uuid PRIMARY KEY,
  reference          text NOT NULL UNIQUE,      -- REQ-YYYY-NNNNNN
  organization_id    uuid NOT NULL REFERENCES organization(id), -- owning tenant
  client_org_id      uuid REFERENCES organization(id), -- subject of the case
  case_type_id       uuid NOT NULL REFERENCES case_type(id),
  subject            text NOT NULL,
  description         text,
  payload            jsonb NOT NULL DEFAULT '{}', -- validated against form_schema
  status             case_status NOT NULL DEFAULT 'DRAFT',
  priority           case_priority NOT NULL DEFAULT 'NORMAL',
  owner_user_id      uuid REFERENCES "user"(id),
  created_by         uuid NOT NULL REFERENCES "user"(id),
  submitted_at       timestampz,
  sla_due_at         timestampz,
  sla_paused_ms      bigint NOT NULL DEFAULT 0, -- accumulated ON_HOLD / AWAITING_INFO
  first_response_at  timestampz,
  resolved_at        timestampz,
  closed_at          timestampz,
  search_vector       tsvector,
  created_at         timestampz NOT NULL DEFAULT now(),
  updated_at         timestampz NOT NULL DEFAULT now(),
  deleted_at         timestampz
);

```

```

CREATE INDEX ix_case_org_status ON "case"(organization_id, status) WHERE deleted_at IS NULL;
CREATE INDEX ix_case_owner_open ON "case"(owner_user_id, sla_due_at)
WHERE status NOT IN ('CLOSED','ARCHIVED','REJECTED');
CREATE INDEX ix_case_client ON "case"(client_org_id);
CREATE INDEX ix_case_search ON "case" USING gin(search_vector);

CREATE TABLE case_status_history (
  id          uuid PRIMARY KEY,
  case_id     uuid NOT NULL REFERENCES "case"(id) ON DELETE CASCADE,
  from_status case_status,
  to_status   case_status NOT NULL,
  reason      text,
  actor_id    uuid NOT NULL REFERENCES "user"(id),
  occurred_at timestampz NOT NULL DEFAULT now()
);

CREATE TABLE comment (
  id          uuid PRIMARY KEY,
  case_id     uuid NOT NULL REFERENCES "case"(id) ON DELETE CASCADE,
  author_id   uuid NOT NULL REFERENCES "user"(id),
  body        text NOT NULL,
  is_internal boolean NOT NULL DEFAULT false, -- hidden from client/partner roles
  created_at  timestampz NOT NULL DEFAULT now(),
  edited_at   timestampz,
  deleted_at  timestampz
);
CREATE INDEX ix_comment_case ON comment(case_id, created_at DESC);

```

6.5 Workflow Tables

```

CREATE TABLE task (
  id          uuid PRIMARY KEY,
  case_id     uuid NOT NULL REFERENCES "case"(id) ON DELETE CASCADE,
  organization_id uuid NOT NULL REFERENCES organization(id),
  type        text NOT NULL, -- REVIEW | COLLECT_DOCS | APPROVE | ...
  title       text NOT NULL,
  description  text,
  assignee_user_id uuid REFERENCES "user"(id),
  assignee_role_id uuid REFERENCES role(id), -- queue assignment when no named user
  status      task_status NOT NULL DEFAULT 'OPEN',
  due_at      timestampz,
  started_at  timestampz,
  completed_at timestampz,
  sequence    int NOT NULL DEFAULT 0, -- ordering inside the process
  blocked_by_id uuid REFERENCES task(id),
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now()
);
CREATE INDEX ix_task_assignee_open ON task(assignee_user_id, due_at)
WHERE status IN ('OPEN','IN_PROGRESS');

CREATE TABLE approval (
  id          uuid PRIMARY KEY,
  task_id     uuid NOT NULL REFERENCES task(id) ON DELETE CASCADE,
  level       int NOT NULL DEFAULT 1,
  required_role_id uuid REFERENCES role(id),
  state       approval_state NOT NULL DEFAULT 'PENDING',
  decided_by  uuid REFERENCES "user"(id),
  decided_at  timestampz,
  comment     text,
  delegated_to uuid REFERENCES "user"(id),
  expires_at  timestampz,
  created_at  timestampz NOT NULL DEFAULT now(),
  UNIQUE (task_id, level)
);

CREATE TABLE process_template (
  id          uuid PRIMARY KEY,
  case_type_id uuid NOT NULL REFERENCES case_type(id),
  name        text NOT NULL,
  steps       jsonb NOT NULL, -- ordered task definitions + transition guards
  version     int NOT NULL DEFAULT 1,

```

```

    is_active      boolean NOT NULL DEFAULT true,
    UNIQUE (case_type_id, version)
);

CREATE TABLE escalation_rule (
    id              uuid PRIMARY KEY,
    case_type_id    uuid REFERENCES case_type(id),
    trigger         text NOT NULL,      -- SLA_50 | SLA_80 | SLA_BREACH | NO_ACTIVITY
    threshold_hours int,
    action          text NOT NULL,      -- NOTIFY | REASSIGN | RAISE_PRIORITY | ESCALATE
    target_role_id  uuid REFERENCES role(id),
    is_active       boolean NOT NULL DEFAULT true
);

```

6.6 Document, Finance, Messaging and Audit Tables

```

CREATE TABLE document (
    id              uuid PRIMARY KEY,
    organization_id uuid NOT NULL REFERENCES organization(id),
    case_id         uuid REFERENCES "case"(id) ON DELETE CASCADE,
    parent_id       uuid REFERENCES document(id),  -- previous version
    version         int NOT NULL DEFAULT 1,
    filename        text NOT NULL,
    mime_type       text NOT NULL,
    size_bytes      bigint NOT NULL,
    checksum_sha256 text NOT NULL,
    storage_key     text NOT NULL,      -- object storage path, never a public URL
    classification  doc_class NOT NULL DEFAULT 'INTERNAL',
    category        text,              -- CONTRACT | ID | TAX | INVOICE | OTHER
    expires_at      date,              -- for certificates and licences
    scan_status     text NOT NULL DEFAULT 'PENDING', -- PENDING|CLEAN|INFECTED
    uploaded_by     uuid NOT NULL REFERENCES "user"(id),
    created_at      timestampz NOT NULL DEFAULT now(),
    deleted_at      timestampz
);
CREATE INDEX ix_document_case ON document(case_id) WHERE deleted_at IS NULL;
CREATE INDEX ix_document_expiry ON document(expires_at) WHERE expires_at IS NOT NULL;

CREATE TABLE signature_request (
    id              uuid PRIMARY KEY,
    document_id     uuid NOT NULL REFERENCES document(id),
    provider        text NOT NULL,
    provider_ref    text NOT NULL,
    signers         jsonb NOT NULL,      -- [{email,name,order,signed_at}]
    status          text NOT NULL DEFAULT 'SENT',
    completed_at    timestampz,
    evidence_key    text,              -- certificate of completion in storage
    created_at      timestampz NOT NULL DEFAULT now()
);

CREATE TABLE invoice (
    id              uuid PRIMARY KEY,
    organization_id uuid NOT NULL REFERENCES organization(id),
    client_org_id   uuid NOT NULL REFERENCES organization(id),
    case_id         uuid REFERENCES "case"(id),
    number          text NOT NULL UNIQUE, -- INV-YYYY-NNNNNN, gapless per year
    status          invoice_status NOT NULL DEFAULT 'DRAFT',
    currency        char(3) NOT NULL,
    subtotal        numeric(14,2) NOT NULL DEFAULT 0,
    tax_total       numeric(14,2) NOT NULL DEFAULT 0,
    total           numeric(14,2) NOT NULL DEFAULT 0,
    amount_paid     numeric(14,2) NOT NULL DEFAULT 0,
    issue_date      date,
    due_date        date,
    pdf_key         text,
    issued_by       uuid REFERENCES "user"(id),
    created_by      uuid NOT NULL REFERENCES "user"(id),
    created_at      timestampz NOT NULL DEFAULT now(),
    updated_at      timestampz NOT NULL DEFAULT now(),
    CONSTRAINT invoice_total_positive CHECK (total >= 0),
    CONSTRAINT invoice_paid_bounded CHECK (amount_paid <= total)
);

```

```

CREATE INDEX ix_invoice_client_status ON invoice(client_org_id, status);
CREATE INDEX ix_invoice_overdue ON invoice(due_date) WHERE status IN ('ISSUED', 'PARTIALLY_PAID');

CREATE TABLE invoice_line (
  id          uuid PRIMARY KEY,
  invoice_id  uuid NOT NULL REFERENCES invoice(id) ON DELETE CASCADE,
  label       text NOT NULL,
  quantity    numeric(12,3) NOT NULL DEFAULT 1,
  unit_price  numeric(14,2) NOT NULL,
  tax_rate    numeric(5,2) NOT NULL DEFAULT 0,
  line_total  numeric(14,2) NOT NULL,
  sequence    int NOT NULL DEFAULT 0
);

CREATE TABLE payment (
  id          uuid PRIMARY KEY,
  invoice_id  uuid NOT NULL REFERENCES invoice(id),
  provider    text NOT NULL,          -- stripe | flutterwave | bank | cash
  provider_ref text,                  -- external transaction id
  method      text NOT NULL,          -- CARD | MOBILE_MONEY | TRANSFER | CASH
  amount      numeric(14,2) NOT NULL CHECK (amount > 0),
  currency    char(3) NOT NULL,
  status      payment_status NOT NULL DEFAULT 'INITIATED',
  paid_at     timestampz,
  reconciled_at timestampz,
  recorded_by uuid REFERENCES "user"(id),
  raw_payload jsonb,                  -- provider webhook body, for disputes
  created_at  timestampz NOT NULL DEFAULT now(),
  UNIQUE NULLS NOT DISTINCT (provider, provider_ref)
);

CREATE TABLE notification (
  id          uuid PRIMARY KEY,
  recipient_id uuid NOT NULL REFERENCES "user"(id) ON DELETE CASCADE,
  organization_id uuid REFERENCES organization(id),
  channel      notif_channel NOT NULL,
  template_code text NOT NULL,
  payload      jsonb NOT NULL DEFAULT '{}',
  subject      text,
  body         text,
  resource_type text, resource_id uuid, -- deep link target
  status      notif_status NOT NULL DEFAULT 'QUEUED',
  provider_ref text,
  error       text,
  sent_at     timestampz,
  read_at     timestampz,
  created_at  timestampz NOT NULL DEFAULT now()
);
CREATE INDEX ix_notif_unread ON notification(recipient_id, created_at DESC)
  WHERE read_at IS NULL AND channel = 'IN_APP';

CREATE TABLE audit_event (
  id          bigserial PRIMARY KEY,
  occurred_at  timestampz NOT NULL DEFAULT now(),
  actor_user_id uuid REFERENCES "user"(id),
  effective_user_id uuid REFERENCES "user"(id), -- set during impersonation
  organization_id uuid REFERENCES organization(id),
  action       text NOT NULL,          -- case.transition, invoice.issue, ...
  resource_type text NOT NULL,
  resource_id  text,
  before      jsonb,
  after       jsonb,
  ip          inet,
  user_agent  text,
  request_id  text,
  outcome     text NOT NULL DEFAULT 'SUCCESS' -- SUCCESS | DENIED | ERROR
);
CREATE INDEX ix_audit_resource ON audit_event(resource_type, resource_id, occurred_at DESC);
CREATE INDEX ix_audit_actor   ON audit_event(actor_user_id, occurred_at DESC);
-- audit_event is append-only: REVOKE UPDATE, DELETE ON audit_event FROM app_role;

```

6.7 Retention and Deletion

Data	Active retention	Then	Trigger
Case and related records	Until closure + 5 years	Archived to cold storage, then anonymised	Nightly retention job
Documents	Until closure + 5 years, or the legal period for the category if longer	Deleted from object storage; metadata kept	Nightly retention job
Audit events	7 years	Exported to cold storage; never deleted while a case is open	Monthly export job
Notifications	13 months	Hard deleted	Nightly
Sessions and refresh tokens	Until expiry + 30 days	Hard deleted	Hourly
Deactivated user accounts	3 years after deactivation	Personal fields anonymised, audit references preserved	On request or automatically

ERASURE A data-subject erasure request **MUST** anonymise the user record and their contact rows, while preserving audit events and financial documents — legal retention overrides erasure for those, and the response to the requester says so explicitly.

7. API Design Conventions

One REST API serves the web application, the mobile application and third parties. There is no privileged internal API: the consoles use exactly the same endpoints and the same authorisation rules as an external integrator would.

7.1 Base Rules

Concern	Rule
Base URL	<code>https://api.businesshub.app/v1</code> . The major version is in the path and only changes on a breaking change.
Transport	HTTPS only, TLS 1.2 minimum. HTTP requests are refused, not redirected.
Media type	<code>application/json; charset=utf-8</code> . File bytes never transit the API — see the upload flow in 8.6.
Naming	Plural, kebab-case collection nouns (<code>/case-types</code>). Field names are camelCase in JSON and snake_case in the database; the mapping lives in one serializer layer.
Dates	ISO 8601 with an explicit offset, always UTC on the wire (<code>2026-08-16T09:14:00Z</code>). The client renders in the user's timezone.
Money	An object: <code>{ "amount": "4850.00", "currency": "EUR" }</code> . The amount is a string to avoid float rounding in JavaScript.
Tenancy	The <code>X-Organization-Id</code> header is required on every authenticated call. A token with a single membership MAY omit it; the server then resolves the default membership.
Idempotency	An <code>Idempotency-Key</code> header is REQUIRED on payment creation and invoice issuance, and honoured on every other mutating endpoint. Keys are retained for 24 hours.
Compression	gzip and brotli are supported on responses over 1 KB.

7.2 Response Envelope

```
// Single resource – 200 OK
{
  "data": { "id": "01917f...", "reference": "REQ-2026-004821", "status": "ESCALATED" },
  "meta": { "requestId": "b6f2...", "timestamp": "2026-08-16T09:14:00Z" }
}

// Collection – 200 OK, cursor paginated
{
  "data": [ { "id": "..." }, { "id": "..." } ],
  "meta": {
    "requestId": "b6f2...",
    "pagination": { "limit": 25, "nextCursor": "eyJpZCI6...", "hasMore": true },
    "totalCount": 248 // omitted when the count is expensive; never guessed
  }
}

// Error – 4xx / 5xx
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "The request payload is invalid.",
    "details": [
      { "field": "dueDate", "rule": "isAfter", "message": "Must be in the future." }
    ],
    "requestId": "b6f2..."
  }
}
```

}

7.3 Status Codes and Error Codes

HTTP	Error code	Meaning and client behaviour
400	BAD_REQUEST	Malformed JSON or an unparsable parameter. Do not retry unchanged.
401	AUTH_REQUIRED	Missing or expired access token. Refresh once, then sign the user out.
401	MFA_REQUIRED	Credentials are valid but a TOTP challenge is outstanding. Present the challenge.
403	PERMISSION_DENIED	Authenticated but not authorised. Hide the affordance in the UI; do not retry.
403	ORG_FORBIDDEN	The active organisation header does not match a membership. Force organisation re-selection.
404	NOT_FOUND	The resource does not exist, or exists in another tenant. Never disclose which.
409	CONFLICT	Version conflict, duplicate unique key, or idempotency-key reuse with a different body.
409	INVALID_TRANSITION	The requested status change is not allowed from the current state. Reload the record.
422	VALIDATION_FAILED	Well-formed but semantically invalid. <code>details[]</code> maps one-to-one to form fields.
423	ACCOUNT_LOCKED	Too many failed sign-ins. Show the unlock time.
429	RATE_LIMITED	Quota exceeded. Honour Retry-After ; back off exponentially with jitter.
500	INTERNAL_ERROR	Unexpected server fault. Show a generic message and the <code>requestId</code> for support.
503	SERVICE_UNAVAILABLE	A dependency is down or the system is in maintenance. Retry after the stated delay.

LEAK PREVENTION Error messages returned to clients MUST NOT contain SQL, stack traces, internal hostnames or provider payloads. The `requestId` is the only handle a user gives to support, and it MUST appear in the logs of every service that touched the request.

7.4 Filtering, Sorting, Pagination and Search

```
GET /v1/cases
  ?status=IN_PROGRESS,ESCALATED      # CSV = OR within a field
  &priority=HIGH
  &ownerUserId=01917f...
  &clientOrgId=01917a...
  &createdAt[gte]=2026-07-01T00:00:00Z
  &createdAt[lt]=2026-08-01T00:00:00Z
  &slaBreached=true
  &q=contract+renewal                # full-text over reference, subject, description
  &sort=-slaDueAt,createdAt          # leading minus = descending
  &limit=25&cursor=eyJpZCI6...
  &include=owner,clientOrg,taskCount # bounded expansion, max 3 relations
```

- Pagination is cursor-based. Offset pagination is not offered — it degrades and it double-counts under concurrent writes.

- `limit` defaults to 25 and is capped at 100 for interactive endpoints, 1000 for export endpoints.
- `totalCount` is returned only when the filtered set is under 10 000 rows; above that it is omitted rather than estimated.
- Unknown query parameters are rejected with 422 — silently ignoring a typo hides bugs for months.
- Every list endpoint applies the tenant filter before any user-supplied filter. There is no code path where a filter can widen the visible set.

7.5 Concurrency, Webhooks and Rate Limits

Mechanism	Specification
Optimistic locking	Mutating endpoints return an <code>ETag</code> . A PATCH SHOULD send <code>If-Match</code> ; a mismatch returns 409 <code>CONFLICT</code> with the current representation so the client can merge.
Outbound webhooks	Signed with HMAC-SHA256 over the raw body plus a timestamp, in <code>X-BH-Signature</code> . At-least-once delivery, retried at 1 m, 5 m, 30 m, 2 h, 12 h. Consumers MUST be idempotent on <code>eventId</code> .
Inbound webhooks	Payment and e-signature callbacks verify the provider signature before parsing. An unverified body is logged and discarded, never processed.
Rate limits	Anonymous 30 req/min per IP; authenticated 600 req/min per user; export endpoints 10 req/min; sign-in 10 req/15 min per account. Limits are returned in <code>X-RateLimit-Limit</code> , <code>-Remaining</code> , <code>-Reset</code> .
Long operations	Anything over 5 seconds (exports, bulk imports, PDF batches) returns 202 with a job id; the client polls <code>GET /v1/jobs/{id}</code> or listens on the notification stream.

8. Endpoint Catalogue

The OpenAPI document in `packages/contracts` is the executable version of this catalogue and is the source of truth for request and response shapes. Where the two disagree, the OpenAPI file wins and this table is a defect. Every row names the permission the guard enforces.

8.1 Authentication and Session

Method & path	Purpose	Permission	Notes
POST <code>/v1/auth/register</code>	Self-service client account creation	public	Rate limited; sends a verification email; never reveals whether the address already exists.
POST <code>/v1/auth/verify-email</code>	Confirm an email address	public	Single-use token, 24 h lifetime.
POST <code>/v1/auth/login</code>	Exchange credentials for tokens	public	Returns 401 <code>MFA_REQUIRED</code> plus a challenge id when MFA is enrolled.
POST <code>/v1/auth/mfa/verify</code>	Complete the TOTP challenge	public + challenge	Accepts a recovery code as an alternative.
POST <code>/v1/auth/refresh</code>	Rotate the refresh token	public + refresh token	Reuse detection revokes the family.
POST <code>/v1/auth/logout</code>	Revoke the current session	authenticated	Idempotent.
POST <code>/v1/auth/password/forgot</code>	Request a reset link	public	Always returns 204, whatever the address.
POST <code>/v1/auth/password/reset</code>	Set a new password	public + token	Revokes every session of the user.
GET <code>/v1/auth/me</code>	Current user, memberships and effective permissions	authenticated	The web app calls this once per session and caches it.
GET <code>/v1/auth/sessions</code>	List active sessions	authenticated	—
DELETE <code>/v1/auth/sessions/{id}</code>	Revoke a session	authenticated / <code>user.update</code>	Own sessions always; others need the permission.
POST <code>/v1/auth/mfa/enrol</code>	Start TOTP enrolment	authenticated	Returns the provisioning URI and recovery codes once.
DELETE <code>/v1/auth/mfa</code>	Disable MFA	authenticated	Refused for roles where MFA is mandatory.

8.2 Organisations, Users and Roles

Method & path	Purpose	Permission
GET <code>/v1/organizations</code>	List organisations visible to the caller	<code>org.read</code>
POST <code>/v1/organizations</code>	Create a client or partner organisation	<code>org.create</code>
GET <code>/v1/organizations/{id}</code>	Organisation detail, including compliance status	<code>org.read</code>
PATCH <code>/v1/organizations/{id}</code>	Update identity, address, tier, account manager	<code>org.update</code>
POST <code>/v1/organizations/{id}/suspend</code>	Suspend, with a mandatory reason	<code>org.suspend</code>

Method & path	Purpose	Permission
POST /v1/organizations/{id}/reactivate	Lift a suspension	org.suspend
GET /v1/organizations/{id}/members	List memberships and roles	user.read
POST /v1/organizations/{id}/invitations	Invite a user by email with a role	user.invite
POST /v1/invitations/{token}/accept	Accept an invitation	public + token
DELETE /v1/memberships/{id}	Remove a user from an organisation	user.deactivate
PATCH /v1/memberships/{id}	Change a member's role	role.assign
GET /v1/users	Directory search within the active organisation	user.read
GET /v1/users/{id}	User profile	user.read
PATCH /v1/users/{id}	Update profile, locale, timezone, notification preferences	user.update / self
POST /v1/users/{id}/deactivate	Disable an account and revoke its sessions	user.deactivate
GET /v1/roles	System and custom roles with their permission sets	role.read
POST /v1/roles	Create a custom role	role.manage
PATCH /v1/roles/{id}	Edit a custom role (system roles are immutable)	role.manage

8.3 CRM

Method & path	Purpose	Permission
GET /v1/clients	Client organisations with open-case and balance summaries	crm.read
GET /v1/partners	Partner organisations with compliance and contract summaries	crm.read
GET /v1/clients/{id}/overview	The 360° payload behind the record screen	crm.read
POST /v1/contacts	Create a contact under an organisation	crm.create
PATCH /v1/contacts/{id}	Update a contact	crm.update
DELETE /v1/contacts/{id}	Logically delete a contact	crm.delete
POST /v1/contacts/{id}/consents	Record or withdraw a communication consent	crm.update
POST /v1/contacts/{id}/portal-access	Grant portal access, creating the user and membership	user.invite
GET /v1/crm/segments	Saved segments and their live counts	crm.read
POST /v1/crm/export	Queue a CSV/XLSX export of the current filter	crm.export

8.4 Cases and Requests

Method & path	Purpose	Permission
GET /v1/case-types	Active case types with their intake JSON Schema	authenticated

Method & path	Purpose	Permission
GET /v1/cases	Filtered, sorted, paginated case list	case.read.*
POST /v1/cases	Create a case in DRAFT ; validates payload against the type schema	case.create
GET /v1/cases/{id}	Case detail with tasks, documents, comments and history	case.read.*
PATCH /v1/cases/{id}	Update subject, description, payload, priority	case.update
POST /v1/cases/{id}/submit	DRAFT → SUBMITTED ; starts the SLA clock	case.create
POST /v1/cases/{id}/qualify	Set type and priority; SUBMITTED → QUALIFIED	case.update
POST /v1/cases/{id}/assign	Set the owner; instantiates the process template	case.assign
POST /v1/cases/{id}/transition	Any guarded status change, with a reason	case.transition
POST /v1/cases/{id}/hold	Pause the SLA clock with a reason	case.transition
POST /v1/cases/{id}/resume	Resume the clock, adding the paused interval	case.transition
POST /v1/cases/{id}/close	Close a resolved case	case.close
POST /v1/cases/{id}/reopen	Reopen within 30 days of closure, with a reason	case.reopen
GET /v1/cases/{id}/history	Status history and audit trail for the case	case.read.*
GET /v1/cases/{id}/comments	Comment thread, internal comments filtered by role	case.read.*
POST /v1/cases/{id}/comments	Post a comment; isInternal requires an internal role	case.update
POST /v1/cases/bulk/assign	Assign up to 100 cases in one call	case.assign
POST /v1/cases/export	Queue an export of the current filter	report.export

8.5 Workflow — Tasks and Approvals

Method & path	Purpose	Permission
GET /v1/tasks	Task list; assignee=me is the personal inbox	task.read
POST /v1/tasks	Create an ad-hoc task on a case	task.create
PATCH /v1/tasks/{id}	Edit title, description or due date	task.create
POST /v1/tasks/{id}/claim	Take a queue task and become its assignee	task.read
POST /v1/tasks/{id}/reassign	Move a task to another user, with a reason	task.reassign
POST /v1/tasks/{id}/complete	Mark done; unblocks the next step in the process	task.complete
POST /v1/tasks/{id}/block	Mark blocked with a cause; stops the task clock	task.complete
GET /v1/approvals	Approvals awaiting the caller, oldest first	approval.read
POST /v1/approvals/{id}/decide	Approve or reject with a comment	approval.decide
POST /v1/approvals/{id}/delegate	Delegate to a peer at the same level or above	approval.decide

Method & path	Purpose	Permission
POST /v1/approvals/{id}/override	Override a pending level; always audited and notified	approval.override
GET /v1/process-templates	Templates by case type and version	admin.reference.manage
POST /v1/process-templates	Publish a new template version	admin.reference.manage
GET /v1/escalation-rules	Escalation rules	admin.reference.manage
PUT /v1/escalation-rules/{id}	Update a rule; takes effect on the next sweep	admin.reference.manage

8.6 Documents

File bytes never pass through the API process. Upload is a three-step handshake so that a 40 MB contract does not occupy a Node worker, and so that a failed or abandoned upload leaves no orphan record in a usable state.

- 1) POST /v1/documents/upload-sessions

```
{ "caseId": "...", "filename": "contract_v3.pdf",
  "mimeType": "application/pdf", "sizeBytes": 1258291,
  "checksumSha256": "9f86d0..." }
```

→ 201 { "uploadId": "...", "url": "https://s3...", "fields": {...}, "expiresIn": 600 }
- 2) PUT/POST the bytes directly to the pre-signed URL (browser or mobile → storage)
- 3) POST /v1/documents/upload-sessions/{uploadId}/complete
→ 201 { "id": "...", "version": 1, "scanStatus": "PENDING" }
The server verifies size and checksum against the object, then queues the virus scan.
A document with scanStatus != CLEAN is not downloadable and is not counted as supplied.

Method & path	Purpose	Permission
GET /v1/documents	List by case, organisation, category or expiry window	document.read
POST /v1/documents/upload-sessions	Open an upload session and get a pre-signed URL	document.upload
POST /v1/documents/upload-sessions/{id}/complete	Finalise, verify checksum, queue the scan	document.upload
GET /v1/documents/{id}/download-url	Time-limited download URL, 5 minutes, single use	document.download
POST /v1/documents/{id}/versions	Upload a new version, keeping the chain	document.upload
PATCH /v1/documents/{id}	Change category, classification or expiry date	document.classify
DELETE /v1/documents/{id}	Logical deletion; the object is removed by the retention job	document.delete
POST /v1/documents/{id}/signature-requests	Send for electronic signature	document.sign.request
GET /v1/signature-requests/{id}	Signature progress and evidence file	document.read
POST /v1/webhooks/esign	Provider callback	signature-verified

8.7 Finance

Method & path	Purpose	Permission
GET /v1/invoices	Invoice list with ageing buckets	finance.read
POST /v1/invoices	Create a draft invoice with lines	invoice.create
PATCH /v1/invoices/{id}	Edit a draft only; issued invoices are immutable	invoice.create
POST /v1/invoices/{id}/issue	Assign the gapless number, freeze totals, generate the PDF, notify	invoice.issue
POST /v1/invoices/{id}/void	Void with a reason; requires a credit note if partly paid	invoice.void
GET /v1/invoices/{id}/pdf-url	Time-limited link to the rendered invoice	finance.read
POST /v1/invoices/{id}/payment-intents	Start an online payment; returns the provider handoff	finance.read (client)
POST /v1/payments	Record a manual payment (transfer, cash, cheque)	payment.record
POST /v1/payments/{id}/refund	Full or partial refund	payment.refund
POST /v1/webhooks/payments	Provider callback; the only path that may set SUCCEEDED	signature-verified
GET /v1/finance/reconciliation	Unmatched payments against open invoices	finance.reconcile
POST /v1/finance/reconciliation/match	Match a payment to an invoice	finance.reconcile
GET /v1/finance/statements	Per-organisation statement of account	finance.read

8.8 Communication, Reporting and Administration

Method & path	Purpose	Permission
GET /v1/notifications	In-app notification feed for the caller	authenticated
POST /v1/notifications/{id}/read	Mark read	authenticated
POST /v1/notifications/read-all	Mark the whole feed read	authenticated
GET /v1/notifications/stream	Server-sent events for live badge and toast updates	authenticated
GET /v1/notification-preferences	Per-channel, per-event preferences	authenticated
PUT /v1/notification-preferences	Update preferences	authenticated
GET /v1/notification-templates	Templates with variables and locales	notification.template.manage
PUT /v1/notification-templates/{code}	Edit a template; a preview render is mandatory before saving	notification.template.manage
POST /v1/devices	Register a mobile push token	authenticated
GET /v1/reports/dashboard	KPI tiles for the caller's role and organisation	report.read
GET /v1/reports/{code}	Parameterised report data (JSON)	report.read
POST /v1/reports/{code}/export	Queue a PDF or XLSX export	report.export

Method & path	Purpose	Permission
GET /v1/jobs/{id}	Status and result link of an asynchronous job	authenticated (owner)
GET /v1/admin/reference/{set}	Reference data: case types, categories, tax rates, calendars	admin.reference.manage
PUT /v1/admin/reference/{set}	Update a reference set	admin.reference.manage
GET /v1/admin/settings	Organisation settings: branding, SLA calendar, numbering	org.settings.manage
PUT /v1/admin/settings	Update settings	org.settings.manage
GET /v1/admin/integrations	Integration status and last successful call	admin.integration.manage
POST /v1/admin/integrations/{code}/test	Send a test call and return the raw result	admin.integration.manage
GET /v1/admin/audit-events	Audit search by actor, resource, action or window	admin.audit.read
POST /v1/admin/audit-events/export	Queue a signed audit export	admin.audit.read
GET /v1/health	Liveness; no authentication, no data	public
GET /v1/health/ready	Readiness: database, Redis, storage and queue depth	public

9. Functional Specification by Module

Each module below states its purpose, its numbered functional requirements, and the user stories that carry testable acceptance criteria. Requirement priority uses MoSCoW: **M** must have in release 1, **S** should have, **C** could have if capacity allows, **W** won't have in release 1.

Acceptance criteria are written so that they map one-to-one onto automated tests. A story is not done until every criterion listed against it has a passing test in the suite named in section 17.

M1 — Accounts, Organisations and Access

Purpose: establish who the actor is, which organisation they are acting for, and what they are allowed to do. Every other module depends on this one being correct; it is built and hardened first.

Functional Requirements

ID	Requirement	Priority
FR-1.1	A visitor can create an account with an email address and a password, and must verify the address before signing in.	M
FR-1.2	A user can sign in, and is challenged for a TOTP code when MFA is enrolled.	M
FR-1.3	MFA enrolment is mandatory for Manager, Administrator and Super-administrator; those roles cannot be assigned to a user who has not enrolled.	M
FR-1.4	A user can reset a forgotten password through a single-use, time-limited link, which revokes all existing sessions.	M
FR-1.5	A user holding several memberships can switch active organisation without signing out, and the whole interface re-scopes.	M
FR-1.6	An administrator can invite a user by email with a chosen role; the invitation expires after 14 days and can be resent or revoked.	M
FR-1.7	An administrator can deactivate a user, which immediately revokes every session and reassigns their open tasks to the queue.	M
FR-1.8	An administrator can create custom roles from the permission catalogue; system roles cannot be edited or deleted.	S
FR-1.9	A user can view and revoke their own active sessions, with device, location and last-seen time.	S
FR-1.10	A super-administrator can impersonate a user for support; the session is read-only and every action records both identities.	S
FR-1.11	A partner administrator can manage users inside their own organisation only.	M
FR-1.12	Sign-in, sign-out, permission change, role change and impersonation each write an audit event.	M
FR-1.13	Single sign-on via an external OIDC provider.	W

User Stories

ID	User story	Acceptance criteria
US-1.1	As a visitor, I want to create an account so that I can submit a request without calling anyone.	Given a valid, unused email and a 12-character password, when I submit the form, then a user is created with status INVITED and a verification email is sent within 60 seconds.

ID	User story	Acceptance criteria
		Given an email that already exists, when I submit, then the response is identical to the success case and no account is created — enumeration is not possible. Given an unverified account, when I attempt to sign in, then I receive 403 with a clear instruction to check my inbox and an option to resend. Given a password under 12 characters or present in the breach list, when I submit, then I receive 422 with the failing rule named.
US-1.2	As a manager, I want MFA enforced on my account so that a stolen password is not enough to reach client data.	Given my role requires MFA and I have not enrolled, when I sign in, then I am routed to enrolment and can reach no other screen. Given I scan the provisioning QR code and submit a valid code, then MFA is enabled and ten recovery codes are displayed exactly once. Given MFA is enabled, when I sign in with a correct password, then I receive 401 MFA_REQUIRED with a challenge id, and tokens are issued only after a valid TOTP or recovery code. Given I submit an expired TOTP code, then I receive 401 and the failed-attempt counter increments. Given I use a recovery code, then that code is consumed, cannot be reused, and I am emailed that a recovery code was used.
US-1.3	As a user of two organisations, I want to switch context so that I see one client's data at a time.	Given memberships in organisations A and B, when I select B, then every subsequent request carries B and lists contain no row belonging to A. Given I switch, then cached queries for A are discarded, not merely hidden. Given I forge the header with an organisation I have no membership in, then I receive 403 ORG_FORBIDDEN and an audit event with outcome DENIED is written.
US-1.4	As an administrator, I want to invite a colleague with a role so that they can start work immediately.	Given I submit an email and a role I am allowed to assign, then an invitation is created, expiring in 14 days, and an email is sent. Given the invitee accepts, then a user and a membership are created and they land on the dashboard for that role. Given the invitation is expired or revoked, when it is opened, then a clear message is shown and no membership is created. Given I attempt to assign a role with more permissions than my own, then I receive 403 and no invitation is created.
US-1.5	As an administrator, I want to deactivate a departing employee so that access ends the same hour.	Given an active user with open tasks, when I deactivate them, then every session is revoked within 30 seconds and their access token is rejected on next use. Given deactivation, then their open tasks return to the role queue and their manager is notified with the list. Given deactivation, then the user record is retained for audit and is not physically deleted.

M2 — CRM: Clients, Partners and Contacts

Purpose: hold one authoritative record per external organisation, with its people, its history and its compliance state, so that no one has to ask a colleague what the status of a relationship is.

Functional Requirements

ID	Requirement	Priority
FR-2.1	Create, read, update and logically delete client and partner organisations, with duplicate detection on tax identifier and legal name.	M

ID	Requirement	Priority
FR-2.2	Manage contacts under an organisation, with exactly one primary contact enforced by the database.	M
FR-2.3	A 360° record showing open cases, contracts, documents, invoices, balance, compliance checklist and activity trail.	M
FR-2.4	Record and withdraw communication consents per contact and per channel, with the timestamp and the source of the consent.	M
FR-2.5	A compliance checklist per organisation, driven by required document categories and their expiry dates.	M
FR-2.6	Grant portal access to a contact, which creates the user and the membership in one action.	M
FR-2.7	Full-text and faceted search across organisations and contacts, returning results in under 500 ms at 100 000 records.	M
FR-2.8	Saved segments with live counts, reusable as filters and as notification audiences.	S
FR-2.9	Export the current filtered set to CSV or XLSX, asynchronously, with the export itself audited.	S
FR-2.10	Merge two duplicate organisations, moving cases, documents and invoices to the survivor and keeping a redirect.	C

User Stories

ID	User story	Acceptance criteria
US-2.1	As an agent, I want the platform to warn me about duplicates so that we stop creating three records for one company.	Given I enter a tax identifier that already exists in the same country, then I receive 409 with a link to the existing record. Given I enter a legal name with a similarity above 0.85 to an existing record, then a non-blocking warning lists the candidates before I can save. Given I confirm the record is genuinely distinct, then it is created and the decision is recorded in the audit trail.
US-2.2	As an account manager, I want a single screen showing everything about a partner so that I can prepare a call in two minutes.	Given a partner with cases, contracts, invoices and documents, when I open the record, then all summaries render in one request in under 1.5 seconds at the 95th percentile. Given a document category is required and missing or expired, then the compliance checklist marks it, and the record header shows a warning state. Given I lack finance permissions, then the transactions panel is absent from the response, not merely hidden by CSS.
US-2.3	As a compliance officer, I want expiring documents surfaced before they lapse so that files never go stale.	Given a document with an expiry date 30 days away, then a task is created for the account manager and a notification is sent. Given the expiry date passes with no replacement, then the organisation compliance status becomes AT_RISK and appears in the daily digest. Given a valid replacement is uploaded, then the checklist item returns to valid and the task closes automatically.
US-2.4	As a client contact, I want my consent choices respected so that I stop receiving messages I declined.	Given I withdraw marketing consent, then no message of category MARKETING is queued for me, verified by an integration test on the notification dispatcher. Given I withdraw all consents, then transactional messages about my own cases are still delivered, since they are not marketing. Given any consent change, then the previous value, the new value, the timestamp and the actor are recorded.

M3 — Requests and Cases

Purpose: the operational heart of the platform. Every interaction with a client or partner becomes a case with a reference, an owner, a deadline and a history.

Functional Requirements

ID	Requirement	Priority
FR-3.1	Case types are configurable: label, intake form as JSON Schema, required document categories, SLA in hours, number of approval levels, client visibility.	M
FR-3.2	A case receives a unique human-readable reference <code>REQ-YYYY-NNNNNN</code> , gapless within the year, generated inside the creating transaction.	M
FR-3.3	The intake form renders dynamically from the case type schema, on both web and mobile, with client-side and server-side validation from the same schema.	M
FR-3.4	Attachments can be added during creation and at any later point, subject to the document rules of M5.	M
FR-3.5	Submission starts the SLA clock, computed against the organisation's business calendar, not wall-clock hours.	M
FR-3.6	Status transitions follow the state machine in section 10 and are refused otherwise, with 409 <code>INVALID_TRANSITION</code> .	M
FR-3.7	Assignment to a named user or a role queue, manually or by a round-robin rule on the case type.	M
FR-3.8	A comment thread with internal-only comments invisible to client and partner roles at the API level.	M
FR-3.9	Complete status history with actor, timestamp and reason, exposed to the client as a simplified progress timeline.	M
FR-3.10	Bulk assignment of up to 100 cases, executed transactionally per case with a per-case result report.	S
FR-3.11	Reopening within 30 days of closure, with a mandatory reason, returning the case to <code>IN_PROGRESS</code> and restarting a shortened SLA.	S
FR-3.12	Merging duplicate cases and linking related cases.	C

User Stories

ID	User story	Acceptance criteria
US-3.1	As a client, I want to submit a request from the portal so that I stop sending email attachments into the void.	Given a case type with a required field left empty, when I submit, then I receive 422 and the field is highlighted with the message from the schema. Given all required fields and documents are supplied, when I submit, then the case moves <code>DRAFT</code> → <code>SUBMITTED</code>, a reference is issued and shown, and an acknowledgement email arrives within 60 seconds. Given submission, then the SLA due date is computed from the case type and my organisation's calendar, and is visible to me as a plain date. Given a required document category is missing, then submission is refused with a list naming each missing category.
US-3.2	As an agent, I want a work queue sorted by urgency so that I always know what to pick up next.	Given cases assigned to me, when I open my queue, then they are sorted by SLA due date ascending with breached cases pinned at the top. Given a case is within 20% of its SLA remaining, then it renders in the at-risk state on web and mobile using the same threshold constant. Given I filter by status and priority, then the filter is reflected in the URL and can be shared with a colleague, who sees the same set subject to their own permissions.

ID	User story	Acceptance criteria
		Given 10 000 cases in the organisation, then the first page returns in under 400 ms at the 95th percentile.
US-3.3	As a manager, I want illegal status changes to be impossible so that reporting can be trusted.	Given a case in DRAFT, when a transition to RESOLVED is requested, then the API returns 409 INVALID_TRANSITION and the state is unchanged. Given any accepted transition, then a <code>case_status_history</code> row is written in the same transaction as the status update — never one without the other. Given a transition requiring a reason (hold, reject, reopen, escalate), when the reason is missing, then the request is refused with 422. Given concurrent transition requests on the same case, then exactly one succeeds and the other receives 409 with the current state.
US-3.4	As a client, I want to see progress without asking so that I stop phoning for updates.	Given my case is in progress, when I open it in the portal, then I see a simplified timeline with completed steps, the current step and the expected date. Given internal comments exist, then they are absent from my API response entirely. Given the case needs something from me, then the portal shows a prominent action banner naming exactly what is required and by when.

M4 — Workflow: Tasks, Approvals and Escalations

Purpose: turn a case into scheduled work, route decisions to people entitled to make them, and make sure nothing stalls silently.

Functional Requirements

ID	Requirement	Priority
FR-4.1	A process template per case type defines an ordered list of tasks with types, owners, due offsets and guards.	M
FR-4.2	Assigning a case instantiates the active template version; the version is recorded on the case so later template edits do not rewrite history.	M
FR-4.3	Tasks may be assigned to a named user or to a role queue that any holder of that role can claim.	M
FR-4.4	Multi-level approvals: each level names a required role; a level may only be decided by a user at or above its approval level.	M
FR-4.5	Completing a task unblocks the next task in the sequence automatically and notifies its assignee.	M
FR-4.6	Escalation rules fire at configurable thresholds (50%, 80%, 100% of SLA, or inactivity) with actions: notify, reassign, raise priority, escalate the case.	M
FR-4.7	Reminders are sent before a due date; the schedule is configurable per task type and per organisation.	M
FR-4.8	Delegation of an approval to a peer at the same level or above, for a bounded period, fully audited.	S
FR-4.9	A manager can override a pending approval level; the override is audited and the original approver is notified.	S
FR-4.10	A visual process-template editor.	C

User Stories

ID	User story	Acceptance criteria
US-4.1	As an agent, I want the next step to appear automatically so that I do not have to remember the procedure.	Given a case is assigned, then the tasks of the active template version are created with due dates derived from the SLA and the template offsets. Given I complete task 2 of 5, then task 3 changes from BLOCKED to OPEN and its assignee is notified within 60 seconds. Given a template is edited afterwards, then cases already instantiated keep the version they started with.
US-4.2	As a manager, I want approvals routed by level so that nobody approves above their authority.	Given an approval at level 2 and I hold approval level 1, when I attempt to decide it, then I receive 403 and the approval remains PENDING. Given I hold level 2 or above, when I approve with a comment, then the approval records my identity, the decision, the comment and the timestamp, and the next level opens. Given the final level approves, then the case transitions to APPROVED and the requester is notified. Given any level rejects, then the case transitions to REJECTED, remaining levels are cancelled, and the reason is mandatory.
US-4.3	As a manager, I want escalations to fire on their own so that a breach is never a surprise.	Given a case reaches 80% of its SLA with no resolution, then an escalation rule fires once — never repeatedly — and notifies the owner and their manager. Given the SLA is breached, then the case transitions to ESCALATED, the priority is raised one step, and it appears in the manager's escalation panel. Given the case is on hold, then SLA thresholds do not fire and the paused interval is excluded from elapsed time. Given the escalation sweep runs, then it is idempotent: running it twice in the same minute produces one notification, not two.
US-4.4	As an agent going on leave, I want to delegate my approvals so that work continues.	Given I delegate to a peer for a date range, then approvals falling in that range appear in their queue and mine shows them as delegated. Given the range ends, then delegation stops automatically without manual cleanup. Given a delegate decides, then the audit event records both the delegate and the original approver.

M5 — Documents

Purpose: store every file once, know which version is current, control who may open it, and prove who opened it.

Functional Requirements

ID	Requirement	Priority
FR-5.1	Upload through pre-signed URLs; file bytes never pass through the API process.	M
FR-5.2	Server-side verification of size and SHA-256 checksum before a document is marked complete.	M
FR-5.3	Virus scanning; a document that is not CLEAN cannot be downloaded and does not satisfy a required-document rule.	M
FR-5.4	Versioning with a parent chain; the newest version is current and older versions remain retrievable.	M
FR-5.5	Four classification levels driving access, with CONFIDENTIAL restricted to holders of <code>document.classify</code> plus the case owner.	M
FR-5.6	Category and expiry date per document, feeding the compliance checklist of M2.	M

ID	Requirement	Priority
FR-5.7	Time-limited, single-use download URLs of at most five minutes; every issuance is audited with the requesting user.	M
FR-5.8	Accepted types: PDF, JPEG, PNG, DOCX, XLSX, CSV. Maximum 50 MB per file; the type is verified by magic bytes, not by extension.	M
FR-5.9	Electronic signature request with ordered signers, status tracking and storage of the completion certificate.	S
FR-5.10	Document templates with variable substitution to generate contracts and letters.	C
FR-5.11	Full-text search inside PDF content.	W

User Stories

ID	User story	Acceptance criteria
US-5.1	As a client, I want to upload a large contract from the portal so that I stop hitting email size limits.	Given a 40 MB PDF, when I upload it, then the browser sends the bytes straight to object storage and the API process memory does not grow. Given the upload completes, then the server verifies size and checksum and rejects the document if either mismatches. Given a file whose magic bytes contradict its extension, then it is rejected with 422 naming the detected type. Given the scan returns INFECTED, then the object is quarantined, the uploader is notified, and no download URL can be issued.
US-5.2	As an agent, I want to replace a document with a corrected version so that the case always shows the right one.	Given an existing document, when I upload a new version, then version increments, the parent link is set, and the case shows only the newest by default. Given a previous version, then it remains retrievable and is labelled with its version number and upload date. Given a versioned upload, then the original is never overwritten in object storage.
US-5.3	As a compliance officer, I want confidential documents restricted so that sensitive files stay with the people who need them.	Given a document classified CONFIDENTIAL, when a user without <code>document.classify</code> who is not the case owner requests it, then they receive 404 — the existence of the document is not disclosed. Given any download URL issuance, then an audit event records the user, the document, the time and the IP. Given a download URL, then it expires after five minutes and cannot be used a second time.

M6 — Finance

Purpose: issue invoices that survive an audit, take payment through several channels, and reconcile what arrived against what was owed.

Functional Requirements

ID	Requirement	Priority
FR-6.1	Draft invoices with lines, quantities, unit prices, per-line tax rates, and totals computed server-side only.	M
FR-6.2	Issuing assigns a gapless number <code>INV-YYYY-NNNNNN</code> , freezes the invoice, generates the PDF and notifies the client.	M
FR-6.3	An issued invoice is immutable. Corrections happen through a credit note, never through an edit.	M

ID	Requirement	Priority
FR-6.4	Online payment through the configured provider, with the provider handoff started server-side and never trusting a client-reported outcome.	M
FR-6.5	Manual payment recording for bank transfer, cash and cheque, with the recording user captured.	M
FR-6.6	Payment status is set to SUCCEEDED only by a signature-verified provider webhook or by an explicit manual entry with <code>payment.record</code> .	M
FR-6.7	Partial payments; the invoice moves to PARTIALLY_PAID and to PAID only when the sum of successful payments equals the total.	M
FR-6.8	Automatic overdue marking and a configurable dunning schedule of reminders.	M
FR-6.9	A reconciliation screen listing unmatched payments against open invoices, with manual matching.	S
FR-6.10	Statement of account per organisation with ageing buckets at 30, 60 and 90 days.	S
FR-6.11	Refunds, full or partial, restricted to <code>payment.refund</code> and always leaving both records visible.	S
FR-6.12	Multi-currency with a recorded conversion rate.	C
FR-6.13	Accounting-package export (CSV per ledger).	C

User Stories

ID	User story	Acceptance criteria
US-6.1	As a finance officer, I want invoice numbers with no gaps so that the sequence stands up to an audit.	Given two invoices issued concurrently, then both receive distinct consecutive numbers and neither transaction leaves a gap. Given issuance fails after the number is allocated, then the whole transaction rolls back and the number is reused by the next issuance. Given the year changes, then the sequence restarts at 000001 with the new year in the prefix.
US-6.2	As a client, I want to pay online and see it reflected at once so that I do not have to send proof of payment.	Given an issued invoice, when I start a payment, then the provider handoff is created server-side with an idempotency key and the amount comes from the database, never from the browser. Given the provider webhook reports success and its signature verifies, then the payment becomes SUCCEEDED, the invoice balance updates, a receipt is generated and I am notified. Given a webhook with an invalid signature, then it is logged and discarded and no financial record changes. Given the same webhook is delivered three times, then the payment is recorded once — matching on the provider reference. Given the browser is closed mid-payment, then the webhook still settles the invoice correctly.
US-6.3	As a controller, I want segregation of duties so that one person cannot invent and settle an invoice alone.	Given I created an invoice, when I attempt to issue it, then I am refused unless a second user with <code>invoice.issue</code> acts, or the organisation has fewer than three finance users. Given the small-organisation override applies, then the action proceeds and an audit event of type SOD_OVERRIDE is written and surfaced in the monthly control report. Given I recorded a payment, then I cannot also approve its refund.
US-6.4	As a finance officer, I want overdue invoices chased automatically so that collections do not depend on my memory.	Given an invoice passes its due date unpaid, then its status becomes OVERDUE within the next nightly run and the client contact is notified.

ID	User story	Acceptance criteria
		Given the dunning schedule is 7, 14 and 30 days, then exactly one reminder is sent at each point and none after payment. Given a payment arrives, then pending reminders for that invoice are cancelled, not merely suppressed at send time.

M7 — Communication and Notifications

Purpose: tell the right person the right thing on the channel they chose, once — and be able to prove it was sent.

Functional Requirements

ID	Requirement	Priority
FR-7.1	Four channels: in-app, email, SMS, mobile push. Every notification is persisted regardless of channel.	M
FR-7.2	Templates identified by code, with named variables, per-locale bodies, and a mandatory preview render before saving.	M
FR-7.3	Per-user, per-event-type, per-channel preferences, with a system-wide default that a user may narrow but not widen beyond policy.	M
FR-7.4	Transactional notifications about a user's own cases are always delivered and are not subject to marketing consent.	M
FR-7.5	Asynchronous delivery through a queue with retry and exponential backoff; a failed send never blocks the originating request.	M
FR-7.6	Delivery status tracking: QUEUED, SENT, DELIVERED, FAILED, READ, with the provider reference stored.	M
FR-7.7	A live in-app feed with an unread badge, updated over server-sent events.	M
FR-7.8	Deep links: every notification carries a resource type and id so that a tap opens the right screen on web and mobile.	M
FR-7.9	Quiet hours per user; non-urgent notifications are held and batched into a digest.	S
FR-7.10	A daily or weekly digest summarising activity, respecting the same preferences.	S
FR-7.11	Two-way messaging between client and case officer inside the case.	S

User Stories

ID	User story	Acceptance criteria
US-7.1	As an agent, I want to be told the moment a case is assigned to me so that I do not discover it after the deadline.	Given a case is assigned, then a notification is queued within 5 seconds and delivered on every channel I have enabled for that event. Given I have disabled SMS for assignments, then no SMS is sent, and the in-app record still exists. Given delivery fails, then it is retried with backoff up to five times and the final failure is visible to an administrator. Given I tap the push notification, then the mobile app opens the case detail directly, not the home screen.
US-7.2	As an administrator, I want to edit a template safely so that a bad variable never reaches a client.	Given I edit a template, then I must render a preview with sample data before the save button becomes available.

ID	User story	Acceptance criteria
		Given the template references an unknown variable, then saving is refused with the variable named. Given a locale is missing, then the fallback locale is used and the gap is listed on the template screen.
US-7.3	As a user, I want quiet hours so that a routine reminder does not wake me.	Given quiet hours 21:00–07:00 in my timezone, when a non-urgent notification is generated at 23:00, then it is held and delivered at 07:00. Given an urgent notification (SLA breach, security alert), then it is delivered immediately regardless of quiet hours. Given several notifications are held, then they arrive as one digest, not as a burst.

M8 — Reporting and Dashboards

Purpose: answer the questions management actually asks — how much work is open, how fast are we, where is the money — from the same database that runs operations, with no manual spreadsheet in between.

Functional Requirements

ID	Requirement	Priority
FR-8.1	A role-specific dashboard: KPI tiles, one trend chart, one distribution chart, and an action queue.	M
FR-8.2	Core operational KPIs: open cases, cases at SLA risk, breached cases, average handling time, first-response time, pending approvals.	M
FR-8.3	Core financial KPIs: issued, collected, outstanding, overdue, and ageing buckets.	M
FR-8.4	Filters on period, organisation, case type, owner and priority, with the filter state encoded in the URL.	M
FR-8.5	Asynchronous export to PDF and XLSX, returning a job id, with the resulting file stored and access-controlled.	M
FR-8.6	Every report respects the caller's permissions and tenancy — a report is never a way around access control.	M
FR-8.7	Saved views, private or shared with a role.	S
FR-8.8	Scheduled reports emailed on a cron expression to a named audience.	S
FR-8.9	Pre-aggregated daily snapshots so that dashboards do not scan the transactional tables at peak.	S
FR-8.10	A custom report builder.	W

User Stories

ID	User story	Acceptance criteria
US-8.1	As a director, I want the dashboard to load quickly so that I actually use it in a meeting.	Given 100 000 cases and 250 000 audit events, when I open the dashboard, then it responds in under 2 seconds at the 95th percentile. Given the same filters within 60 seconds, then the response is served from cache and marked as such in <i>meta</i>. Given a KPI cannot be computed, then the tile shows an explicit error state — never a silent zero.
US-8.2	As a manager, I want to export what I see so that I can attach it to a board pack.	Given a filtered view, when I export, then I receive a job id immediately and a notification when the file is ready.

ID	User story	Acceptance criteria
		Given the export completes, then the file contains exactly the rows the filter produced, with the filter printed in the header. Given I lack <code>report.export</code>, then the export control is absent and a direct API call returns 403.
US-8.3	As a partner administrator, I want reports limited to my own organisation so that I never see another partner's figures.	Given I request any report, then every row is scoped to my organisation, enforced in the query, not filtered after the fact. Given I tamper with the organisation parameter, then I receive 403 ORG_FORBIDDEN and the attempt is audited.

M9 — Administration, Audit and Integrations

Purpose: let the platform be configured without a deployment, and make every sensitive action reconstructable afterwards.

Functional Requirements

ID	Requirement	Priority
FR-9.1	Reference data managed through the interface: case types, document categories, tax rates, business calendars, priorities.	M
FR-9.2	Organisation settings: branding, locale, currency, SLA calendar, invoice numbering, dunning schedule.	M
FR-9.3	An audit trail covering authentication, permission changes, case transitions, document downloads, financial actions and exports.	M
FR-9.4	Audit search by actor, resource, action, outcome and time window, with a signed export.	M
FR-9.5	The audit trail is append-only; the application database role has no UPDATE or DELETE grant on it.	M
FR-9.6	Integration configuration with a connection test that shows the raw provider response.	M
FR-9.7	Outbound webhook subscriptions with HMAC signing, retry policy and a delivery log.	S
FR-9.8	A feature-flag mechanism so that a module can be enabled per organisation.	S
FR-9.9	A maintenance mode returning 503 with a stated window, while health endpoints stay live.	S
FR-9.10	Public content management for the marketing pages.	C

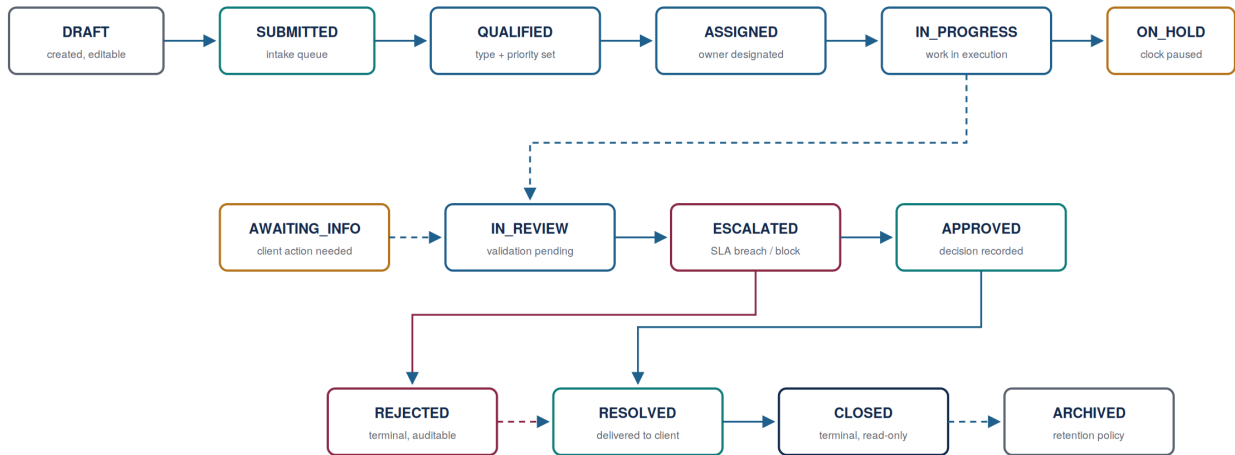
User Stories

ID	User story	Acceptance criteria
US-9.1	As an auditor, I want to reconstruct what happened to a case so that a dispute can be settled from evidence.	Given a case reference, when I search the audit trail, then I see every action in order with actor, timestamp, IP and before/after values. Given an administrator attempts to modify an audit row, then the database refuses the statement — this is verified by an automated test. Given I export the audit set, then the export is itself audited and the file carries a checksum.
US-9.2	As an administrator, I want to add a case type without a release so that the business is not blocked by the sprint calendar.	Given I define a case type with an intake schema, then it becomes available to new cases immediately, with no restart. Given the schema is invalid JSON Schema, then saving is refused with the parse error shown.

ID	User story	Acceptance criteria
		Given existing cases of that type, then editing the schema does not retroactively invalidate them.
US-9.3	As an administrator, I want to test an integration before switching it on so that clients are not the ones who discover it is broken.	Given credentials for the payment provider, when I run the test, then I see the raw response and a clear success or failure verdict. Given the test fails, then the integration cannot be marked active. Given secrets are displayed, then they are masked except for the last four characters, both in the interface and in the logs.

10. State Machines and Process Rules

REQUEST / CASE LIFECYCLE — STATE MACHINE



Solid arrows: nominal transitions. Dashed: conditional transitions (SLA timer, client response, manager override). Every transition writes an audit event and may fire a notification.

Figure 3 — Case lifecycle: permitted transitions

10.1 Case Transition Table

This table is the specification. Implement it as a declarative map in `modules/cases/transitions.ts` and drive both the guard and the tests from that single map — never as a chain of if-statements in a controller.

From	To	Permission	Guard conditions	Side effects
DRAFT	SUBMITTED	<code>case.create</code>	All required fields and document categories supplied.	Reference issued, SLA clock starts, acknowledgement sent.
SUBMITTED	QUALIFIED	<code>case.update</code>	Case type and priority set.	SLA recomputed for the confirmed type.
SUBMITTED	REJECTED	<code>case.transition</code>	Reason mandatory.	Client notified with the reason; terminal.
QUALIFIED	ASSIGNED	<code>case.assign</code>	Owner is an active member with the required role.	Process template instantiated; owner notified.
ASSIGNED	IN_PROGRESS	<code>case.transition</code>	Owner has started at least one task.	First-response timestamp recorded.
IN_PROGRESS	AWAITING_INFO	<code>case.transition</code>	A request to the client names what is needed.	SLA paused; client notified with a deadline.
AWAITING_INFO	IN_PROGRESS	<code>case.transition</code>	Requested items supplied, or the agent resumes manually.	SLA resumed; paused interval added to <code>sla_paused_ms</code> .
IN_PROGRESS	ON_HOLD	<code>case.transition</code>	Reason mandatory.	SLA paused; escalation rules suspended.
ON_HOLD	IN_PROGRESS	<code>case.transition</code>	—	SLA resumed.
IN_PROGRESS	IN_REVIEW	<code>case.transition</code>	All mandatory tasks complete.	Approval chain opened at level 1.

From	To	Permission	Guard conditions	Side effects
IN_REVIEW	APPROVED	<code>approval.decide</code>	Every approval level approved.	Case ready for delivery; requester notified.
IN_REVIEW	REJECTED	<code>approval.decide</code>	Any level rejects; reason mandatory.	Remaining levels cancelled; requester notified.
Any active	ESCALATED	<code>case.transition</code> or system	SLA breach, block, or manual escalation with a reason.	Priority raised one step; manager notified; appears in escalation panel.
ESCALATED	IN_PROGRESS	<code>case.transition</code>	Blocking cause resolved.	Escalation closed with an outcome note.
APPROVED	RESOLVED	<code>case.transition</code>	Deliverable attached or outcome recorded.	Client notified; satisfaction survey queued.
RESOLVED	CLOSED	<code>case.close</code>	No open task, no unpaid mandatory invoice.	Case becomes read-only; SLA metrics frozen.
CLOSED	IN_PROGRESS	<code>case.reopen</code>	Within 30 days; reason mandatory.	New shortened SLA; owner and client notified.
CLOSED	ARCHIVED	system	Retention period elapsed.	Moved to cold storage; documents purged per policy.

10.2 Invoice and Payment State Rules

Transition	Trigger	Constraint
DRAFT → ISSUED	<code>POST /invoices/{id}/issue</code>	Requires at least one line, a due date, and a second actor under segregation of duties. Number allocated inside the transaction.
ISSUED → PARTIALLY_PAID	Successful payment below the total	Sum of successful payments recomputed server-side; never incremented from a webhook amount alone.
ISSUED / PARTIALLY_PAID → PAID	Sum of successful payments equals the total	Exact equality on <code>numeric</code> ; no floating-point tolerance.
ISSUED / PARTIALLY_PAID → OVERDUE	Nightly job, due date passed	Reversible: a later payment moves it to PAID, not back to ISSUED.
Any → DISPUTED	Client raises a dispute case	Dunning suspended while disputed; a case is created and linked.
Any → CREDITED	Credit note issued	The original invoice is never edited; the credit note references it.
DRAFT → VOID	<code>POST /invoices/{id}/void</code>	Allowed on a draft at any time; on an issued invoice only when nothing is paid, and always with a reason.
INITIATED → SUCCEEDED	Signature-verified webhook, or manual entry	A client-reported outcome is never sufficient. Duplicate provider references are rejected by a unique constraint.

10.3 SLA Computation

- The SLA due date is computed from the case type's `sla_hours` applied to the organisation's business calendar: working days, working hours and public holidays. A 24-hour SLA raised at 16:00 on a Friday is due at 11:00 on the following Monday, not on Saturday.
- Time spent in `ON_HOLD` and `AWAITING_INFO` accumulates into `sla_paused_ms` and is excluded from elapsed time.

- Escalation thresholds are evaluated against elapsed working time, not wall-clock time.
- The SLA sweep runs every five minutes, is idempotent, and records the last processed timestamp so that a restart does not re-fire notifications.
- Changing a case type after qualification recomputes the due date from the submission timestamp, and the recomputation is written to the history with both values.

11. Cross-Cutting Business Rules

ID	Rule	Enforced where
BR-01	Every business record carries an organisation, an owner, a status and a time-stamped history.	Schema constraints + base entity
BR-02	Nothing is physically deleted by a user action; deletion sets <code>deleted_at</code> and the record disappears from every default query.	Prisma middleware + query defaults
BR-03	Every mutating action writes an audit event in the same transaction as the change.	Audit interceptor
BR-04	A status change never happens without a permission check and a guard evaluation.	Transition service
BR-05	Amounts, totals and taxes are computed server-side; a client-supplied total is ignored, not validated.	Finance service
BR-06	Uniqueness that matters to the business (invoice number, case reference, primary contact, tax identifier) is enforced by a database constraint, never only in application code.	Schema
BR-07	Any operation touching more than one table runs in one transaction; partial writes are not acceptable.	Service layer
BR-08	Background work is idempotent and safe to replay; every job checks whether its effect already exists.	Worker
BR-09	A user never sees a record from an organisation they do not belong to, including in search results, exports, notifications and error messages.	RLS + tenancy guard
BR-10	Time is stored in UTC and rendered in the user's timezone; no business logic uses the server's local time.	Base entity + client
BR-11	Money never uses a floating-point type at any layer, including in JSON on the wire.	Schema + serializer
BR-12	An action that would breach segregation of duties is refused, or permitted with an explicit, audited override — never silently allowed.	Ability service

12. Web Application Specification

One Next.js application serves two audiences from one codebase and one component library. Internal consoles live under `/app`, the external portal under `/portal`. Route groups keep the layouts and the middleware separate, so a portal user can never render a console shell even if a link leaks.

12.1 Route Map

Route	Audience	Purpose	Guard
<code>/ /features /contact</code>	Public	Marketing pages and the request-intake entry point.	none
<code>/auth/*</code>	Public	Sign in, register, MFA challenge, password reset.	none
<code>/app/dashboard</code>	Internal	Role dashboard: KPI tiles, charts, priority queue.	<code>report.read</code>
<code>/app/cases /app/cases/[id]</code>	Internal	Case list and full case workspace.	<code>case.read.*</code>
<code>/app/tasks /app/approvals</code>	Internal	Personal task inbox and approval queue.	<code>task.read / approval.read</code>
<code>/app/clients/[id] /app/partners/[id]</code>	Internal	CRM 360° records.	<code>crm.read</code>
<code>/app/documents</code>	Internal	Document library with category and expiry filters.	<code>document.read</code>
<code>/app/finance/*</code>	Internal	Invoices, payments, reconciliation, statements.	<code>finance.read</code>
<code>/app/reports/*</code>	Internal	Reports and exports.	<code>report.read</code>
<code>/app/admin/*</code>	Internal	Users, roles, reference data, settings, integrations, audit.	<code>admin.*</code>
<code>/portal/requests /portal/requests/[id]</code>	External	Client and partner request tracking.	<code>membership + case.read.own</code>
<code>/portal/documents /portal/invoices /portal/messages</code>	External	Client self-service.	<code>membership</code>

12.2 Screen Composition Rules

- Every list screen has: a filter bar whose state lives in the URL, a sortable table, a bulk-action bar that appears on selection, an empty state that explains what to do, and a skeleton loading state — not a spinner.
- Every detail screen has: a header with identity and status, a primary action bar, tabbed panels loaded lazily, and an activity timeline.
- Destructive actions require a typed confirmation of the record reference, not a bare "Are you sure?".
- Server-state uses TanStack Query with a 30-second stale time; mutations invalidate precisely the affected keys, never the whole cache.
- Forms are built once from the case type JSON Schema and reused on web and mobile, so validation cannot drift between them.
- A permission the user lacks removes the affordance from the DOM. Disabling it and relying on the API to refuse is not acceptable — it leaks the existence of the capability.

12.3 Accessibility and Internationalisation

Requirement	Specification
Standard	WCAG 2.1 level AA across all screens, verified by axe-core in CI and by one manual audit before release.
Keyboard	Every interactive element reachable and operable by keyboard; a visible focus ring; a skip-to-content link; focus trapped inside modals and returned on close.
Contrast	Minimum 4.5:1 for body text and 3:1 for large text and interface components. Status is never conveyed by colour alone — a label or an icon accompanies it.
Screen readers	Semantic landmarks, labelled form controls, aria-live announcements for asynchronous results, and a page title that changes with the route.
Motion	prefers-reduced-motion disables non-essential animation.
Languages	English and French at launch, with the locale on the user record. Strings live in message catalogues; none are hard-coded in components.
Formats	Dates, numbers and currency formatted through the Intl API for the active locale; no manual string assembly.
Right-to-left	Layout uses logical CSS properties so that an RTL locale can be added without restyling.

12.4 Web Performance Budget

Metric	Target	Measured by
Largest Contentful Paint	≤ 2.5 s on a 4G connection	Lighthouse CI on every pull request
Interaction to Next Paint	≤ 200 ms	Real-user monitoring
Cumulative Layout Shift	≤ 0.1	Lighthouse CI
Initial JavaScript bundle	≤ 200 KB gzipped per route	Bundle-size check, fails the build
Table render, 1 000 rows	≤ 100 ms with virtualisation	Component benchmark

13. Mobile Application Specification

The mobile application is not a smaller copy of the console. It targets the three things people genuinely do away from a desk: approve, capture, and check status.

13.1 Scope

Included	Excluded (use the web application)
Dashboard: pending approvals, my cases, overdue items	Role and permission administration
Case list, case detail, timeline, comments	Invoice creation and editing
Approve / reject with a comment	Reference data and process template configuration
Create a request from a simplified form	Reconciliation and statements
Document capture by camera with edge detection and PDF assembly	Report building and bulk exports
Record a payment and issue a receipt	Audit trail search
Push notifications with deep links	Bulk operations
Biometric unlock and offline queue	Impersonation

13.2 Offline Behaviour

The application is online-first with a bounded offline queue. It does not attempt full offline replication — that would create merge conflicts the business has no rule for.

- Read cache: the last 50 cases, their tasks and their metadata are stored in SQLite via Drift and shown with an explicit "last synchronised" timestamp.
- Write queue: comments, task completions, approvals and captured documents are queued locally with a client-generated UUID used as the idempotency key.
- On reconnection the queue drains in submission order; each item is retried up to five times with exponential backoff.
- A conflict — the case moved on while the device was offline — surfaces as an explicit resolution card showing both states. It is never resolved silently.
- Approvals older than 24 hours in the queue are cancelled and returned to the user for re-confirmation, because the decision context may have changed.
- Captured images are compressed to at most 2 MB before queueing and are uploaded only over the pre-signed URL flow.

13.3 Mobile Platform Requirements

Concern	Requirement
Supported versions	Android 9 (API 28) and above; iOS 15 and above.
Token storage	Access and refresh tokens in Keychain (iOS) and EncryptedSharedPreferences (Android). Never in shared preferences in clear text, never in application logs.

Concern	Requirement
Biometrics	Face ID / Touch ID / fingerprint to unlock a cached session; a failed biometric falls back to full sign-in, never to bypass.
Push	Firebase Cloud Messaging for both platforms; the token is registered per device and removed on sign-out.
Deep links	Every notification carries <code>resourceType</code> and <code>resourceId</code> ; a cold start routes to the target screen after authentication.
Screenshots	Screenshot capture is blocked on screens showing documents classified RESTRICTED or CONFIDENTIAL.
Certificate pinning	The API certificate is pinned, with a backup pin and a documented rotation procedure.
Crash-free rate	≥ 99.5% of sessions, tracked per release in Sentry.
Cold start	≤ 2 seconds to an interactive dashboard on a mid-range device.
Application size	≤ 40 MB download on both stores.

14. External Integrations

Every integration sits behind a port-and-adapter interface in `infrastructure/`. Business code depends on the interface, never on a provider SDK, so that swapping a provider is one adapter and one environment variable.

14.1 Integration Contracts

Integration	Interface	Failure policy
Payment	<code>PaymentProvider: createIntent, capture, refund, verifyWebhook</code>	Circuit breaker after 5 consecutive failures; the client is told payment is temporarily unavailable and the invoice remains unchanged. Never optimistically mark paid.
Email	<code>MailProvider: send, sendTemplate, getStatus</code>	Queued with retry at 1 m, 5 m, 30 m. After the final failure the notification is marked FAILED and appears on the administrator health panel.
SMS	<code>SmsProvider: send, getStatus</code>	Same retry policy. SMS is never the sole channel for a legally significant message.
Push	<code>PushProvider: register, send, unregister</code>	An invalid token is removed from the device table on the first rejection.
E-signature	<code>SignatureProvider: createRequest, getStatus, downloadEvidence</code>	A signature request that receives no callback within 72 hours is flagged for manual follow-up; the document is never assumed signed.
Object storage	<code>StorageProvider: presignUpload, presignDownload, delete, head</code>	A failure blocks the upload and returns 503. A document record is created only after the object is confirmed present.

14.2 Webhook Handling Rules

6. Verify the provider signature over the raw request body **before** parsing the JSON. A body that fails verification is logged with its headers and discarded.
7. Reject any event whose timestamp is more than five minutes old, to close the replay window.
8. Persist the raw event with its provider id before acting on it; the unique constraint on the provider reference makes duplicate delivery harmless.
9. Acknowledge with 200 as soon as the event is persisted; do the business processing in a queued job. A slow handler makes the provider retry and creates duplicates.
10. If processing fails, the job retries; the acknowledgement is not withdrawn.
11. Every webhook, accepted or rejected, produces an audit event.

15. Non-Functional Requirements

15.1 Performance and Capacity

Requirement	Target	Verification
API read latency	p95 ≤ 300 ms, p99 ≤ 800 ms	k6 load test in CI against a seeded dataset
API write latency	p95 ≤ 500 ms	k6
Document upload initiation	p95 ≤ 200 ms (the handshake, not the bytes)	k6
Dashboard aggregation	p95 ≤ 2 s at 100 000 cases	Benchmark against a production-shaped dataset
Concurrent users	500 sustained, 1 500 peak	Load test before each major release
Data volume, year one	250 000 cases, 1 M documents, 5 M audit events	Capacity model reviewed quarterly
Database connections	≤ 20 per API replica, pooled	Configuration + monitoring alert
Background job latency	Notification queued to sent ≤ 60 s at p95	Queue metrics

15.2 Availability and Resilience

Requirement	Specification
Availability target	99.5% monthly, excluding announced maintenance windows — approximately 3 h 40 m of unplanned downtime per month.
Recovery point objective	15 minutes, achieved by continuous WAL archiving.
Recovery time objective	2 hours, rehearsed quarterly with a documented drill report.
Backups	Full nightly, incremental hourly, retained 30 days, stored in a second region. A restore is tested monthly — an untested backup is not a backup.
Degraded mode	If the payment provider is down, everything except payment continues. If Redis is down, the API serves reads and refuses writes rather than losing them.
Zero-downtime deploys	Rolling replacement with health-gated readiness; migrations are expand-then-contract so that old and new code run together during the roll.
Maintenance	Announced at least 48 hours ahead, in-app and by email, executed within the declared window.

15.3 Security Requirements

Area	Requirement
Transport	TLS 1.2 minimum, HSTS with a one-year max-age and preload, no mixed content.
At rest	Database and object storage encrypted with managed keys; MFA secrets and integration credentials encrypted at the application layer with a separate key.
Headers	<code>Content-Security-Policy</code> with no <code>unsafe-inline</code> , <code>X-Content-Type-Options: nosniff</code> , <code>Referrer-Policy: strict-origin-when-cross-origin</code> , <code>Permissions-Policy</code> restricting camera and geolocation to the routes that need them.

Area	Requirement
Injection	All database access through parameterised queries via Prisma; raw SQL requires review and may never interpolate user input.
XSS	No <code>dangerouslySetInnerHTML</code> without sanitisation through an allow-list; user content is escaped by default.
CSRF	Cookie-based sessions use <code>SameSite=Strict</code> plus a double-submit token; the mobile app uses bearer tokens and is not exposed.
Uploads	Type verified by magic bytes, size capped, virus scanned, served from a separate origin with <code>Content-Disposition: attachment</code> .
Secrets	Never in the repository. A secret scanner runs on every push and blocks the merge on a hit.
Dependencies	<code>pnpm audit</code> and Dependabot in CI; a critical vulnerability blocks release; a high is fixed within 7 days.
Testing	Automated OWASP ZAP baseline scan per release; a full external penetration test before go-live and annually thereafter.
Logging	Passwords, tokens, MFA secrets, card data and full document contents are never logged. Log redaction is unit-tested.
Incident response	A documented runbook with severity levels, an on-call rota, a 72-hour breach-notification path, and a post-incident review within five working days.

15.4 Privacy and Compliance

- Data minimisation: a field is collected only if a named requirement in this document uses it.
- Purpose limitation: consents are recorded per channel and per purpose, with the timestamp and the source.
- Right of access: a user can export their own personal data as JSON through a self-service request completed within 30 days.
- Right of erasure: honoured by anonymisation, except where financial and audit retention obligations apply; the response states exactly what was retained and why.
- Data residency: the primary region is configurable per deployment; documents never leave the configured region.
- Processing register: every integration that receives personal data is listed with its purpose, its data categories and its retention period.

16. Observability

Signal	Requirement
Structured logs	JSON only. Every line carries <code>requestId</code> , <code>userId</code> , <code>organizationId</code> , <code>route</code> , <code>durationMs</code> and <code>outcome</code> . Free-text logs without a request id are rejected in code review.
Tracing	OpenTelemetry spans across web → api → database and queue jobs, with the trace id propagated into the browser and the mobile client so a user-reported problem can be pulled up by request id.
Metrics	RED per route (rate, errors, duration); queue depth and job age per queue; database pool saturation; cache hit ratio; SLA breaches per hour.
Business metrics	Cases created, cases closed, average handling time, payment success rate and notification delivery rate, all exported to the same dashboard as the technical metrics.
Alerting	Page on: error rate above 2% for 5 minutes, p95 latency above 1 s for 10 minutes, queue age above 10 minutes, failed backup, certificate within 14 days of expiry, payment webhook failures above 5 in 10 minutes.
Dashboards	One service-health dashboard and one business dashboard, both linked from the runbook and reviewed at the weekly operations meeting.
Retention	Logs 30 days hot and 12 months cold; metrics 13 months; traces 7 days with tail sampling of errors and slow requests.

17. Test Strategy

17.1 Test Pyramid and Coverage Gates

Level	Scope	Tooling	Gate
Unit	Pure logic: SLA calculation, transition guards, permission evaluation, money arithmetic, template rendering.	Jest (TS), flutter_test (Dart)	≥ 80% line coverage; 100% on transitions , permissions and finance modules
Integration	Service plus real PostgreSQL and Redis in containers; no mocked database.	Jest + Testcontainers	Every endpoint in section 8 has at least one happy path and one authorisation-failure test
Contract	Request and response shapes against the OpenAPI document, in both directions.	Dredd / schemathesis	The pipeline fails if the implementation and the contract diverge
End-to-end	The seven journeys in 17.2, on Chromium, Firefox and WebKit.	Playwright	All must pass before a merge to main
Mobile E2E	Approval, capture and offline-queue drain on one Android and one iOS device.	Patrol / integration_test	Nightly, and before every release
Load	Read and write mixes at the targets in 15.1.	k6	Weekly and before every major release
Security	Dependency audit, secret scan, ZAP baseline.	GitHub Actions	Any critical finding blocks the release
Accessibility	Automated rule checks on every screen.	axe-core in Playwright	Zero serious or critical violations

17.2 Mandatory End-to-End Journeys

12. Client registers, verifies their email, submits a request with an attachment, and receives the acknowledgement.
13. Agent qualifies the request, is assigned it, completes two tasks, and sends it for review.
14. Manager approves at two levels and the case is resolved, delivered and closed.
15. A case breaches its SLA, escalates automatically, and appears in the manager's escalation panel with a notification.
16. Finance issues an invoice, the client pays online, the webhook settles it, and the receipt is generated.
17. Partner is invited, accepts, uploads a compliance document, and the checklist turns valid.
18. A user without permission attempts each restricted action and is refused with the correct status code and an audit event.

17.3 Test Data and Environments

- Fixtures are code, not database dumps, and live beside the tests that use them.
- Every test creates its own tenant and cleans up; tests never depend on the order they run in.
- Staging is refreshed weekly from an anonymised production copy: names, emails, phone numbers and document contents are replaced, referential integrity is preserved.
- Payment, email, SMS and signature providers run in sandbox mode outside production. A test that reaches a real provider is a defect.

- A dedicated performance dataset of 250 000 cases is generated by a script kept in the repository, so benchmarks are comparable across months.

18. Delivery, CI/CD and Operations

18.1 Pipeline

```

on: pull_request
  1. install          pnpm install --frozen-lockfile
  2. static           lint · typecheck · format check · secret scan
  3. unit             jest --coverage (fails under threshold)
  4. integration      testcontainers: postgres + redis
  5. contract         validate implementation against packages/contracts
  6. build            docker build api · web · worker (multi-stage, distroless)
  7. e2e             playwright against the composed stack
  8. a11y + lighthouse budgets from 12.4
  9. security         pnpm audit · trivy image scan · zap baseline

on: merge to main
  10. migrate         prisma migrate deploy against staging
  11. deploy staging  rolling, health-gated
  12. smoke          the seven journeys of 17.2 against staging

on: manual promotion (approval required)
  13. migrate production expand-only; contract migrations ship one release later
  14. deploy production canary 10% for 15 min, automatic rollback on error budget burn
  15. verify          smoke tests + dashboard check + release note published

```

18.2 Migration Discipline

- Migrations are forward-only. A mistake is corrected by a new migration, never by editing a merged one.
- Expand then contract: add the new column, backfill in a job, deploy code that writes both, deploy code that reads the new one, and only then drop the old column — in a later release.
- A migration that rewrites a large table runs as a background job with a progress log, not inside the deploy step.
- Every migration is tested against a staging copy of production-sized data before it reaches production.
- Index creation on a live table uses `CREATE INDEX CONCURRENTLY`.

18.3 Release Roadmap

Phase	Duration	Content	Exit criteria
P0 — Foundations	3 weeks	Repository, CI, Docker Compose, schema and migrations, identity, tenancy, RBAC, audit skeleton, design system.	A user can sign in with MFA, switch organisation, and every action is audited. Pipeline green end to end.
P1 — Cases	4 weeks	Case types, dynamic intake, case CRUD, state machine, assignment, comments, history, agent queue.	The first three end-to-end journeys pass. A case cannot reach an illegal state.
P2 — Workflow & documents	4 weeks	Process templates, tasks, approvals, escalations, SLA engine, document upload, versions, classification, scanning.	Journeys 2, 4 and 6 pass. Escalations fire correctly against a simulated clock.
P3 — Portal & notifications	3 weeks	Client and partner portal, notification engine, templates, preferences, in-app feed, email and SMS.	A client completes journey 1 unaided in a usability test with five participants.
P4 — Finance	3 weeks	Invoices, issuance, PDF generation, online payment, webhooks, manual payments, dunning, reconciliation.	Journey 5 passes, including duplicate-webhook and failed-payment cases.

Phase	Duration	Content	Exit criteria
P5 — Mobile	4 weeks	Flutter application: dashboard, cases, approvals, capture, push, offline queue, biometrics.	Mobile end-to-end suite green on both platforms; store builds accepted.
P6 — Reporting & admin	3 weeks	Dashboards, KPIs, exports, reference data, settings, integration console, audit search.	Performance targets met on the 250 000-case dataset.
P7 — Hardening & launch	3 weeks	Penetration test and fixes, accessibility audit, load test, disaster-recovery drill, documentation, training, pilot.	No critical or high finding open; RTO and RPO demonstrated; runbooks signed off.

SIZING Twenty-seven weeks of build for a team of six to eight: one technical lead, two backend, two front-end, one mobile, one QA, and a designer at 50%. The phases are sequential in dependency but P5 can start in parallel with P4 once the case and workflow APIs are stable.

18.4 Definition of Done

- Every acceptance criterion of the story has a passing automated test.
- The endpoint appears in the OpenAPI document with examples, and the generated types are committed.
- Permission and tenancy checks are covered by an explicit negative test.
- An audit event is emitted where section 11 requires one, and a test asserts it.
- The screen passes the axe-core check and has been operated once by keyboard alone.
- Error, empty and loading states exist and are shown in the pull request screenshots.
- Strings are in the message catalogue in both launch languages.
- Observability is in place: the route emits its metric and the log line carries the request id.
- The runbook is updated if the change adds an operational task.
- The product owner has accepted the story on staging, not on a developer machine.

19. Risks and Mitigations

Risk	Impact	Likelihood	Mitigation
Scope creep from "small" client-specific requests	High	High	Case types and process templates are configuration, not code. Anything requiring code goes to the backlog with a written trade-off.
Tenancy leak between organisations	Critical	Low	Two independent layers (application filter plus row-level security) and a mandatory negative test per endpoint.
Payment reconciliation drift	High	Medium	Provider webhooks are the only source of truth for success; a nightly reconciliation job reports any divergence to finance.
SLA engine wrong on calendars and holidays	Medium	Medium	The calendar is data, not code; a property-based test suite covers boundary cases including year ends and holiday clusters.
Document storage cost growth	Medium	High	Lifecycle rules move objects to infrequent-access at 90 days and to archive at 1 year; the retention job actually deletes.
Mobile offline conflicts	Medium	Medium	A bounded queue, idempotency keys, a 24-hour approval expiry, and explicit conflict cards instead of silent merges.
Low adoption because the old spreadsheets still work	High	Medium	Phase the cut-over per department, import historical data, and retire the spreadsheet on a named date agreed in advance.
Key-person dependency on the technical lead	Medium	Medium	Architecture decision records, pairing on every module, and a rule that no module has a single reviewer.

20. Appendices

A. Notification Event Catalogue

Event code	Trigger	Recipients	Default channels
<code>case.submitted</code>	A case moves to SUBMITTED	Submitter, intake queue	In-app, email
<code>case.assigned</code>	An owner is set	New owner	In-app, email, push
<code>case.status_changed</code>	Any status transition	Owner, client contact	In-app, email
<code>case.info_requested</code>	Transition to AWAITING_INFO	Client contact	Email, SMS, push
<code>case.sla_at_risk</code>	80% of SLA elapsed	Owner, manager	In-app, push
<code>case.sla_breached</code>	100% of SLA elapsed	Owner, manager	In-app, email, push
<code>case.resolved</code>	Transition to RESOLVED	Client contact	Email, in-app
<code>task.assigned</code>	A task gains an assignee	Assignee	In-app, push
<code>task.due_soon</code>	Configurable offset before due	Assignee	In-app, push
<code>approval.requested</code>	An approval level opens	Approvers at that level	In-app, email, push
<code>approval.decided</code>	An approval is decided	Requester, case owner	In-app, email
<code>document.expiring</code>	30 days before a document expires	Account manager, organisation admin	Email, in-app
<code>document.rejected</code>	Scan returns INFECTED, or a review rejects	Uploader	Email, in-app
<code>invoice.issued</code>	An invoice is issued	Client billing contact	Email
<code>invoice.overdue</code>	Dunning schedule points	Client billing contact, finance	Email, SMS
<code>payment.received</code>	A payment succeeds	Client contact, finance	Email, in-app
<code>security.new_signin</code>	Sign-in from a new device or location	Account owner	Email
<code>security.mfa_changed</code>	MFA enabled, disabled or recovery code used	Account owner, administrator	Email

B. Standard Audit Actions

<code>auth.login</code> <code>auth.mfa_failed</code> <code>user.created</code> <code>org.created</code> <code>case.created</code> <code>case.closed</code> <code>task.created</code> <code>approval.delegated</code> <code>document.uploaded</code> <code>invoice.created</code> <code>payment.refunded</code> <code>report.exported</code>	<code>auth.login_failed</code> <code>auth.password_reset</code> <code>user.updated</code> <code>org.updated</code> <code>case.submitted</code> <code>case.reopened</code> <code>task.completed</code> <code>approval.overridden</code> <code>document.downloaded</code> <code>invoice.issued</code> <code>payment.webhook</code> <code>admin.reference_changed</code>	<code>auth.logout</code> <code>auth.impersonate</code> <code>user.deactivated</code> <code>org.suspended</code> <code>case.transition</code> <code>case.deleted</code> <code>task.reassigned</code> <code>document.classified</code> <code>invoice.voided</code> <code>finance.reconciled</code> <code>admin.integration_changed</code>	<code>auth.mfa_enrolled</code> <code>auth.session_revoked</code> <code>user.role_changed</code> <code>org.settings_changed</code> <code>case.assigned</code> <code>case.exported</code> <code>approval.decided</code> <code>document.deleted</code> <code>payment.recorded</code> <code>finance.sod_override</code> <code>admin.audit_exported</code>
--	--	---	---

C. Seed Data Loaded in Local and CI Environments

Object	Content
Organisations	One INTERNAL operating company, two CLIENT organisations, one PARTNER organisation.
Roles	The seven system roles of section 5.4, with the permission sets exactly as specified.
Users	One per role, password Password123! , MFA pre-enrolled with a fixed test secret for the roles that require it.
Case types	Contract renewal, onboarding review, invoice dispute, document validation, service complaint — each with an intake schema and a process template.
Cases	Forty cases spread across every status, including two breached and two on hold, so that dashboards are never empty.
Documents	Small sample PDFs and images, one expiring within 30 days, one already expired.
Invoices	Six invoices across DRAFT, ISSUED, PARTIALLY_PAID, PAID, OVERDUE and DISPUTED.
Calendars	A business calendar with 08:00–17:00 working hours, Monday to Friday, and a public-holiday list for the current year.

D. Open Decisions Required Before PO Ends

#	Decision	Owner	Needed by
D-1	Confirm the product name and secure the domain and store identifiers.	Product owner	Start of P0
D-2	Choose the payment provider — the adapter is written against one, and mobile money availability drives the choice.	Finance + product	Start of P4
D-3	Confirm the launch languages and whether a third is planned within a year, which affects the catalogue structure.	Product owner	Start of P1
D-4	Confirm the data-residency region and whether any client contract forbids cross-border processing.	Legal	Start of P0
D-5	Confirm the business calendar and public holidays used for SLA computation.	Operations	Start of P2
D-6	Decide whether e-signature is in release 1 or deferred, since it carries a provider contract.	Product owner	Start of P2
D-7	Agree the historical-data migration scope: which years, which objects, and which are archived rather than imported.	Operations	Start of P6
D-8	Name the pilot department and the date the current spreadsheet is retired.	Executive sponsor	Start of P7

E. Glossary of Technical Terms

Term	Definition
RBAC	Role-based access control: permissions are attached to roles, and roles to users within an organisation.
RLS	Row-level security: a PostgreSQL feature that filters rows inside the database, independently of application code.
Idempotency key	A client-supplied identifier that lets the server recognise a retried request and return the original result instead of acting twice.

Term	Definition
Cursor pagination	Paging by an opaque pointer to the last item seen, rather than by an offset, so results stay stable under concurrent writes.
Pre-signed URL	A temporary, signed link that lets a client read or write an object in storage directly, without the credentials passing through the API.
Expand-and-contract	A migration pattern where the schema first accepts both the old and the new shape, and the old one is removed only after all code has moved.
Circuit breaker	A guard that stops calling a failing dependency for a cooling period, so that one slow provider does not exhaust the whole request pool.
Dead-letter queue	Where a job goes after exhausting its retries, so that it can be inspected instead of disappearing.
p95 / p99	The value below which 95% or 99% of measurements fall — a far more useful latency target than an average.
Error budget	The amount of unreliability the availability target permits; burning it quickly triggers an automatic rollback.